Differential-Time SNN Accelerator: Complete Documentation & Source Code

Repository: `https://github.com/shamekhing/fpga-snn` (cloned at `/home/ifonlyloop/Documents/JKU/TH&S/snn`) **Target FPGA:** Sipeed Tang Primer 20K (Gowin GW2A-LV18PG256C8/I7) **Based on:** *SNN Architecture for Differential Time Encoding Using Decoupled Processing Time* — Windhager, Moser, Lunglmayr (AICAS 2024)

Table of Contents

#	Section	Pages	Files
1	Project Purpose & Scope	1	README.md, .gitignore
2	Implementation Status	1	README.md
3	Repository Layout & Source Map	1	README.md
4	snnTorch Quickstart (with explanation)	1	web reference
5	snnTorch Tutorial 6 — Surrogate Gradients (with explanation)	1	web reference
6	Extending snnTorch — this project	1	—
7	System Overview (with explanation)	1	docs/Theory&Code/01-system-overview.md
8	Differential-Time Encoding Theory (with explanation)	2	docs/Theory&Code/02-theory.md

#	Section	Pages	Files
9	Leaky Integrate-and-Fire Dynamics (with explanation)	1	docs/Theory&Code/02-theory.md
10	Synaptic Input & Training Through Spikes (with explanation)	1	docs/Theory&Code/02-theory.md
11	Time Multiplexing in Hardware (with explanation)	1	docs/Theory&Code/02-theory.md
12	Python Model Module Overview	1	—
13	train_snn.py	4	src/model/train_snn.py
14	encode_mnist.py	2	src/model/encode_mnist.py
15	spike_events.py	1	src/model/spike_events.py
16	snn_ref.py	4	src/model/snn_ref.py
17	quantize.py	1	src/model/quantize.py
18	export_mem.py	1	src/model/export_mem.py
19	test_snn_model.py	2	src/model/test_snn_model.py
20	test_train_snn.py	1	src/model/test_train_snn.py
21	requirements-model.txt	1	src/model/requirements-model.txt
22	RTL Architecture — Central Parameters	1	src/rtl/util/pkg_snn.sv
23	RTL Architecture — Scheduler	2	src/rtl/core/event_sched.sv
24	RTL Architecture — Time Integrator	1	src/rtl/core/time_integrator.sv
25	RTL Architecture — LIF Neuron	1	src/rtl/core/lif_neuron.sv
26		2	src/rtl/core/lif_layer.sv

#	Section	Pages	Files
	RTL Architecture — Time-Multiplexed Hidden Layer		
27	RTL Architecture — Parallel Output Layer	1	src/rtl/core/ lif_layer_parallel.sv
28	RTL Architecture — Spike Counter & Argmax	1	src/rtl/core/ spike_counter.sv, src/ rtl/util/argmax.sv
29	RTL Architecture — Memory ROMs	1	src/rtl/mem/ stream_rom.sv, src/rtl/ mem/weight_rom.sv
30	RTL Architecture — Top-Level	2	src/rtl/top/snn_top.sv
31	Testbenches — Time Integrator	1	src/tb/ tb_time_integrator.sv
32	Testbenches — Event Scheduler	1	src/tb/ tb_event_sched.sv
33	Testbenches — LIF Neuron	1	src/tb/tb_lif_neuron.sv
34	Testbenches — Top-Level	1	src/tb/tb_snn_top.sv
35	Build Scripts	1	src/scripts/run_sim.sh, src/scripts/build.tcl
36	Project Configuration	1	snn.gprj, tang_primer_20k.cst, tang_primer_20k.sdc
37	Version Control	1	.gitignore
38	Verification & Formats	2	docs/Theory&Code/05-formats-and-verification.md
39	Top-Down RTL Workflow	2	docs/Theory&Code/06-top-down-workflow.md
40	Appendix — Complete File Registry	1	all files

Part I — Project Overview

1. Project Purpose & Scope

This repository contains an experimental spiking-neural-network (SNN) project for **MNIST digit classification** using a **Differential-Time Encoding** approach. It combines:

- **Python training / reference-model workflow** using snnTorch for training and algorithmic development.
- **SystemVerilog RTL** intended for FPGA implementation on the **Sipeed Tang Primer 20K** board (Gowin GW2A-18C-LV18PG256C8/I7).

The design is based on the paper *SNN Architecture for Differential Time Encoding Using Decoupled Processing Time* by Windhager, Moser, and Lunglmayr (AICAS 2024). The paper represents spikes as **signed changes in pixel value** (Δx) and **differences between spike times** (Δt). The hardware processes events in strict temporal order; the encoded time is independent of the FPGA clock cycles needed to process them.

1.1 Implementation Status

The following components exist:

- ✅ Python MNIST trainer (`train_snn.py`)
- ✅ Differential-time event encoder (`encode_mnist.py`)
- ✅ Integer event-driven reference model (`snn_ref.py`)
- ✅ Event representation (`spike_events.py`)
- ✅ Quantization helper (`quantize.py`) — template only
- ✅ Memory hex writer (`export_mem.py`) — template only
- ✅ RTL building blocks: `lif_neuron`, `time_integrator`, `event_sched`, `lif_layer`, `lif_layer_parallel`, `spike_counter`, `argmax`, `stream_rom`, `weight_rom`, `snn_top`
- ✅ Icarus Verilog testbenches (time integrator, event scheduler, LIF neuron, top-level)
- ✅ Gowin project file (`snn.gprj`) with RTL, constraints, and pin assignments

What is not** claimed:**

- ❌ FPGA accuracy is not claimed. Local ROM images are non-constant structural test data — not exported trained weights and event streams.
- ❌ The checkpoint-to-ROM exporter is missing.
- ❌ The top-level run window (1024 cycles) is a smoke-test bound, not a real end-of-stream protocol.
- ❌ Gowin synthesis/place-and-route reports are generated by the user's workflow and are excluded from this document (they are implementation artifacts).

1.2 Repository Layout

Path	Purpose
`README.md`	This document's sibling — root README of the repository
`docs/Theory&Code/`	Seven theory/code markdown pages
`src/model/`	Python model: training, encoding, reference, tests
`src/rtl/`	SystemVerilog RTL: package, cores, memories, top level
`src/tb/`	SystemVerilog testbenches for RTL simulation
`src/scripts/`	Icarus Verilog run script, Gowin build TCL template
`src/data/`	MNIST IDX, event streams, weight ROMs (all git-ignored)
`src/constraints/`	Gowin pin constraints (`.cst`) and timing constraints (`.sdc`)
`impl/`	Gowin synthesis, place-and-route, temporary parser output (git-ignored)
`ignore/`	Simulator traces and generated artifacts (git-ignored)

Part II — Background: snnTorch

This section supplements the project's own theory pages with the external snnTorch tutorials. The local theory pages (sections 7–11) explain the project-specific differential-time encoding and integer reference.

4. snnTorch Quickstart (with explanation)

Source: snntorch readthedocs — Quickstart (static non-editable version)

The quickstart demonstrates a minimal SNN for MNIST:

1. **Data loading** — MNIST is loaded with `torchvision` and wrapped in a `DataLoader`. Pixels are resized to 28×28, converted to grayscale, normalized to `[0, 1]`.
2. **Network definition** — `snn.Leaky()` instantiates a leaky integrate-and-fire (LIF) neuron. The `init_hidden=True` flag makes each layer maintain membrane potential as an instance variable.
3. **Surrogate gradient** — The exact spike function is non-differentiable at the threshold. snnTorch provides `snntorch.surrogate.fast_sigmoid()` for a smooth gradient.
4. **Forward pass** — Network is called repeatedly per time step; `init_leaky()` resets hidden state at `t=0`*, then each LIF layer is stepped and spikes are recorded.
5. **Training loop** — Adam optimizer updates weights to minimize cross-entropy over summed output spike counts. `SF.accuracy_rate()` measures prediction accuracy.

Key API patterns used by `train_snn.py`: - `encode_batch(images, threshold=0.05)` converts MNIST rows into differential-time events. - `snntorch.Leaky(beta=0.5, threshold=1.0)` — the project uses β=0.5 and θ=1.0 in floating-point domain. - `CrossEntropyLoss` is applied to `spike_counts = model(inputs).sum(dim=0)`.

5. snnTorch Tutorial 6 — Surrogate Gradient Descent (with explanation)

Source: snntorch readthedocs — Tutorial 6

Tutorial 6 extends the quickstart to a **convolutional SNN (CSNN)** trained with MNIST, introducing **surrogate gradient loss** from Tutorial 5:

- **Surrogate gradient descent:** Because `∂S/∂U = δ(U − θ)` is a Dirac delta, gradient descent cannot propagate through exact spikes. The fix is to replace the Heaviside with a smooth approximation (e.g., `fast_sigmoid()`).
- **Leaky neuron:** Each LIF layer has a decay constant β (default 0.5). `learn_beta=True` allows β itself to be learned.
- **Time-step loop:** The forward pass loops over `num_steps=25`. At each step, input projects to hidden layer, hidden LIF updates, hidden spikes project to output layer, output LIF updates. Returns recorded spikes and membrane states.
- **Loss:** `SF.mse_count_loss(correct_rate=0.8, incorrect_rate=0.2)` measures MSE between target and actual output spike counts.
- **Accuracy:** `SF.accuracy_rate()` counts the percentage of test images where the output neuron with most spikes matches the label.

Code flow in Tutorial 6:

```
forward_pass(net, num_steps, data)
    → for step in range(num_steps):
        cur1 = fc1(x.flatten(1))
        spk1, mem1 = lif1(cur1, mem1)
        cur2 = fc2(spk1)
        spk2, mem2 = lif2(cur2, mem2)
        spk_rec.append(spk2)
    → return torch.stack(spk_rec)
```

Relevance to this project: `train_snn.py` uses the same `snn.Leaky` and `surrogate.fast_sigmoid()` API but feeds differential-time events instead of raw pixels, and uses a fully-connected architecture with 128 hidden neurons.

6. How This Project Extends snnTorch

Aspect	snnTorch Tutorial	This Project
Input	Raw pixel intensity	Differential-time events (`Δx`, `Δt`)
Encoding	Batch-major dense tensor `(time, batch, 784)`	Per-row event streams (`delta_encode`)
Neuron	`snntorch.Leaky(beta=0.5, θ=1.0)`	

Aspect	snnTorch Tutorial	This Project
		Integer reset-to-mod LIF (`POT_W=16`, `W_W=4`, `THETA=16`)
Training	Float32, cross-entropy on spike counts	Float32 with same loss; integer reference for hardware parity
Workflow	Train → eval on CPU	Train → encode → (export ROM) → RTL → simulate
Limitation	—	Python accuracy ≠ FPGA accuracy until quantization/ export is done

Part III — Theory (from local docs)

7. System Overview

Source: `docs/Theory&Code/01-system-overview.md`

Goal: The project classifies MNIST digits with a small SNN and investigates an implementation on a Gowin FPGA. An input image is converted into signed changes over image columns. Those changes become events with a time coordinate, polarity, and input-pixel address.

Logical network:

```
28 x 28 image
    │
    ├─ 28 independent row event streams
    │
    ├─ event scheduler: earliest absolute event time
    │
    ├─ hidden layer: 128 LIF neurons
    │    software: parallel vector update
    │    RTL: one shared arithmetic lane, 128 cycles/event
    │
    ├─ output layer: 10 LIF neurons
    │    parallel in RTL
    │
```

```
    ├─ spike counters
    └─ argmax(counts) → digit 0..9
```

Three representations:

1. **Training representation** — `encode_batch` creates a tensor `(28, batch, 784)`. All row changes at one image column are presented at one time step. This is convenient for PyTorch.
2. **Explicit event representation** — `delta_encode` creates one list per row. Each event has a flattened input index, a delta time, and polarity. This is consumed by `snn_ref.run_image()`.
3. **RTL representation** — ROMs provide stream delta-time words and packed weight rows. The scheduler selects the earliest pending stream time; LIF layers consume one accepted event at a time.

Event-time vs clock-time: An event's encoded time is *not* the number of FPGA clocks used to process it. The FPGA may need several clocks to scan all 28 streams, handle scheduler handshakes, process all 128 hidden neurons, and propagate hidden spikes through the output layer. This separation is the central architectural idea: logical SNN time advances according to event data; implementation time is determined by the hardware pipeline.

Repository flow:

```
MNIST IDX files
   → train_snn.py
   → floating-point checkpoint (mnist_snn.pt)
   → [future quantization/export step]
   → stream.mem and weight ROM files
   → stream_rom / weight_rom
   → snn_top
   → RTL simulation or Gowin implementation
```

Part IV — Background and Theory (continued)

8. Differential-Time Encoding Theory (with explanation)

Source: `docs/Theory&Code/02-theory.md`

For one image row, let `x[c]` be the normalized intensity at column `c`. The change at that column is:

```
Δx[c] = x[c] − x[c−1],  with x[−1] = 0
```

Given threshold `τ = 0.05`, the number of unit events is:

```
n[c] = floor(|Δx[c]| / τ)
```

If `n[c] > 0`, each event has polarity:

```
p[c] = +1  when Δx[c] > 0
       −1  when Δx[c] < 0
```

The first event at column `c` receives a delta time from the previous event. Additional events caused by the same intensity change receive delta time zero. This preserves multiplicity without requiring a second time coordinate.

Each of the 28 rows is an independent stream. The global event sequence is formed by repeatedly selecting the stream with the smallest next absolute time. Equal-time events are combined into one scheduler result using a mask.

Overflow markers: The event format stores delta time in eight bits. `0xFF` is reserved as an overflow marker, so regular deltas are at most 254. A long idle interval is split into one or more time-only markers. An overflow marker:

- advances that stream's absolute time by 255,
- has no input index,
- has polarity zero,
- must not update a neuron.

The representation is defined by `SpikeEvent` in `spike_events.py`; decoding and validation are in `snn_ref.py`.

Key design concepts: - **Differential-time encoding** transforms static pixel values into a temporal event stream. Event temporal position encodes *where* the image intensity changed. - **Signed events** encode both direction (polarity) and magnitude, eliminating the need for an explicit "event rate" encoding. - **Overflow encoding** handles large temporal gaps without requiring 16+ bit time values in the event stream. - **Multiplicity zero-delta** allows a pixel change greater than one threshold unit to emit multiple unit spikes at the same time, preserving information.

9. Leaky Integrate-and-Fire Dynamics (with explanation)

Source: `docs/Theory&Code/02-theory.md`

The integer reference uses a **reset-to-mod** LIF update. For membrane potential `v`, elapsed event time `Δt`, signed synaptic input `I`, and decay shift `s`:

```
v_decay = v >>> (Δt · s)   // right shift by (Δt × s) bits
u       = v_decay + I
```

With positive threshold `θ`:

```
if u ≥ θ:
    spike = +1
    v_next = u − θ
else:
    spike = 0
    v_next = u
```

The project uses `s = 1`, equivalent to a power-of-two decay approximation (right shift by `Δt`).

Optional negative firing uses a lower threshold `θ_low`; it is disabled by default in both the integer reference and the RTL layer instances.

Current RTL parameters: `THETA = 16`. The snnTorch trainer uses `threshold = 1.0` and `beta = 0.5`. Equal beta values do *not* make those models equivalent because the weight scales, arithmetic widths, event grouping, and threshold units differ.

How LIF works in the integer reference: 1. Membrane potential decays as right shift: `v >> (dt × beta_shift)`. With `beta_shift = 1`, this is `v >> dt`. 2. The decayed potential is summed with the synaptic input. 3. If the sum reaches or exceeds the threshold, the neuron fires a positive spike and the membrane resets to `sum − θ`. 4. If `θ_low` is enabled and the sum falls at or below it, the neuron fires a negative spike and resets to `sum − θ_low`. 5. Otherwise, the potential is just stored as-is.

Design notes: - The reset-to-mod operator means the potential can drop below zero, allowing negative spikes. This is essential for an integer event-driven model of signed input. - Right-shifting by elapsed time implements the leak in hardware with a barrel shifter — cheap on an FPGA. - The threshold is *integer*, matching the weight width (`W_W = 4` bits) and the fixed-point representation. This is why `THETA = 16` rather than the snnTorch `1.0`.

10. Synaptic Input & Training Through Spikes (with explanation)

Source: `docs/Theory&Code/02-theory.md`

Hidden layer input: For an input event at flattened address `i` with polarity `p`, hidden neuron `h` receives:

```
I_h = p · W_hidden[i, h]
```

Output layer input: For a hidden spike from neuron `h` with polarity `q`, output neuron `o` receives:

```
I_o = q · W_output[h, o]
```

At one event time, contributions from all streams or hidden neurons active at that time are summed before the LIF update. The integer reference uses NumPy vectors. The RTL uses a polarity signal and a single selected weight row; its exact simultaneous-event behavior must be validated before claiming bit-accurate equivalence.

Training through spikes: The forward pass contains discontinuous threshold operations, so the exact spike function has no useful ordinary derivative. snnTorch uses a surrogate derivative during backpropagation. In `train_snn.py`, `snntorch.surrogate.fast_sigmoid()` is selected and training uses Adam with cross-entropy on the summed output spike counts.

Training objective:

```
counts[o] = Σ_t output_spike[t, o]
prediction  = argmax_o counts[o]
loss        = CrossEntropyLoss(counts, label)
```

Why rate-like readout: This is a rate-like readout over a temporal sequence. It is *not* a proof that the integer or FPGA implementation produces the same counts.

11. Time Multiplexing in Hardware (with explanation)

Source: `docs/Theory&Code/02-theory.md`

The hidden layer has 128 neurons. A fully parallel implementation would replicate decay, addition, threshold comparison, and state storage for every neuron. The RTL instead stores 128 membrane potentials but reuses one arithmetic datapath:

```
accept event
for h = 0 .. 127:
    update potential[h]
    emit spike[h] if threshold crossed
complete event
```

This reduces replicated arithmetic at the cost of up to 128 processing clocks per accepted hidden event. The output layer remains parallel because it has only 10 neurons.

Trade-off: | Design | Arithmetic Replications | Latency per Event | Area | |—|—|—|—| | Fully parallel hidden layer | 128 × LIF datapath | 1 clock per event | Large | | Time-multiplexed hidden layer | 1 × LIF datapath + 128 state registers | 128 clocks per event | Small | | Output layer (parallel) | 10 × LIF datapath | 1 clock per event | Moderate |

Why time-multiplexing: On a mid-range FPGA (GW2A-18C, 18k LUTs), 128 parallel LIF datapaths would consume a significant fraction of the device's logic. A single reusable datapath with state storage reduces this dramatically. The trade-off is increased latency (128 cycles per event), which is acceptable for a classification task.

Part V — Python Model

12. Python Model Module Overview

The Python model provides three layers of computation:

Module	Responsibility
`train_snn.py`	IDX loading, batch encoding, snnTorch model, training, evaluation, checkpoint save
`encode_mnist.py`	Explicit per-row `SpikeEvent` encoding
`spike_events.py`	Event tuple and overflow-code constants
`snn_ref.py`	Integer event-driven hidden/ output reference
`quantize.py`	Standalone float-to-integer quantizer (template)
`export_mem.py`	Generic masked hexadecimal writer (template)

Module	Responsibility
`test_snn_model.py`	Tests for event encoding and the integer reference
`test_train_snn.py`	Tests for MNIST IDX loading and training-input preparation
`requirements-model.txt`	Optional CPU-capable Python dependencies for training

Part VI — Python Model: Source Code and Explanations

13. `train_snn.py`

Purpose: Loads MNIST IDX files, prepares differential-time row inputs, trains an snnTorch network, evaluates it on the test set, and saves a PyTorch checkpoint. This is the **entry point for the training pipeline**.

Key API / signals:

Function	Purpose
`read_idx_images(path)`	Read a MNIST IDX image file into `(count, 28, 28)` uint8 array
`read_idx_labels(path)`	Read a MNIST IDX label file into 1D uint8 array
`load_mnist(data_dir)`	Load the full train/test pair; returns 4 arrays
`encode_batch(images, threshold=0.05)`	Create time-major signed events, output shape `(28, batch, 784)`
`_build_model(args, torch, snntorch)`	Construct the snnTorch `nn.Sequential` with Leaky layers
`train(args)`	Run the training loop with Adam + CrossEntropyLoss
`_evaluate(...)`	Evaluate test accuracy

Full source code:

```
"""Train the MNIST SNN with snnTorch and differential-time row
        inputs."""

from __future__ import annotations
```

```
import argparse
import struct
from pathlib import Path
from typing import TYPE_CHECKING

import numpy as np

if TYPE_CHECKING:
    import torch
    from torch import nn

IMAGE_MAGIC = 2051
LABEL_MAGIC = 2049
IMAGE_ROWS = 28
IMAGE_COLS = 28
HIDDEN_SIZE = 128
OUTPUT_SIZE = 10
INPUT_THRESHOLD = 0.05
NEURON_THRESHOLD = 1.0
NEURON_BETA = 0.5


def read_idx_images(path: str | Path) -> np.ndarray:
    """Read an uncompressed MNIST IDX image file."""
    data = Path(path).read_bytes()
    if len(data) < 16:
        raise ValueError(f"{path}: truncated IDX image header")
    magic, count, rows, columns = struct.unpack(">IIII",
        data[:16])
    if magic != IMAGE_MAGIC:
        raise ValueError(f"{path}: expected image IDX magic
        {IMAGE_MAGIC}, got {magic}")
    expected_size = 16 + count * rows * columns
    if len(data) != expected_size:
        raise ValueError(
            f"{path}: expected {expected_size} bytes for IDX
        images, got {len(data)}"
        )
    if rows != IMAGE_ROWS or columns != IMAGE_COLS:
        raise ValueError(
            f"{path}: expected {IMAGE_ROWS}x{IMAGE_COLS} images,
        got {rows}x{columns}"
        )
    return np.frombuffer(data, dtype=np.uint8, offset=16).reshape(
        count, rows, columns
    )


def read_idx_labels(path: str | Path) -> np.ndarray:
    """Read an uncompressed MNIST IDX label file."""
    data = Path(path).read_bytes()
```

```
    if len(data) < 8:
        raise ValueError(f"{path}: truncated IDX label header")
    magic, count = struct.unpack(">II", data[:8])
    if magic != LABEL_MAGIC:
        raise ValueError(f"{path}: expected label IDX magic
        {LABEL_MAGIC}, got {magic}")
    if len(data) != 8 + count:
        raise ValueError(
            f"{path}: expected {8 + count} bytes for IDX labels,
        got {len(data)}"
        )
    labels = np.frombuffer(data, dtype=np.uint8, offset=8)
    if np.any(labels > 9):
        raise ValueError(f"{path}: labels must be digits in [0,
        9]")
    return labels


def load_mnist(data_dir: str | Path) -> tuple[np.ndarray, ...]:
    """Load and validate the standard train and test IDX file
        pairs."""
    root = Path(data_dir)
    train_images = read_idx_images(root / "train-images.idx3-
        ubyte")
    train_labels = read_idx_labels(root / "train-labels.idx1-
        ubyte")
    test_images = read_idx_images(root / "t10k-images.idx3-ubyte")
    test_labels = read_idx_labels(root / "t10k-labels.idx1-ubyte")
    if len(train_images) != len(train_labels):
        raise ValueError("MNIST training image and label counts
        differ")
    if len(test_images) != len(test_labels):
        raise ValueError("MNIST test image and label counts
        differ")
    return train_images, train_labels, test_images, test_labels


def encode_batch(images: np.ndarray, threshold: float =
        INPUT_THRESHOLD) -> np.ndarray:

        """Create time-major signed row events with one step per
        image column.

    The result has shape (28, batch, 784). Each row emits its
        signed
    intensity changes in parallel at the corresponding column
        time. Changes
    at least ``threshold`` in magnitude produce repeated unit
        events, which
    are represented here by their signed integer amplitude.
    """
    batch = np.asarray(images, dtype=np.float32)
    if batch.ndim != 3 or batch.shape[1:] != (IMAGE_ROWS,
        IMAGE_COLS):
```

```
        raise ValueError(
            f"images must be a 3D array of shape ({IMAGE_ROWS},
        {IMAGE_COLUMNS}, ?)"
        )
    # ... full code continues in the source file
```

Design notes: - **Threshold parameter:** `INPUT_THRESHOLD = 0.05` (5% of full intensity range) controls how many unit events a given Δx produces. - **Float vs integer:** The snnTorch training uses float32 throughout. The RTL reference uses fixed-point integer arithmetic, so training and hardware are not bit-identical until quantization/export. - **Output spike counting:** The model sums spikes over time and uses `CrossEntropyLoss`. This is a rate-based classifier. - **Checkpoint:** Saves `state_dict`, `hidden_size`, `input_threshold`, `neuron_threshold`, `beta`, `epochs`, `seed` to `data/weights/mnist_snn.pt`.

---

14. `encode_mnist.py`

Purpose: Encodes each image row as an independent signed differential-time spike stream. This is the **direct Python-level precursor to the RTL stream ROM**.

Key API / signals:

Symbol	Purpose
`delta_encode(image, threshold=0.05)`	Returns `list[list[SpikeEvent]]` of 28 row streams
`SpikeEvent(input_index, delta_time, polarity)`	Named tuple for a single event
`MAX_REGULAR_DELTA = 254`	Maximum valid delta without overflow
Overflow events	`SpikeEvent(None, 255, 0)` — time-only markers

Full source code:

```
"""Encode MNIST rows as parallel signed differential-time spike
        streams."""

from __future__ import annotations

import math

import numpy as np
from numpy.typing import ArrayLike
```

```
if __package__:
    from .spike_events import MAX_REGULAR_DELTA, SpikeEvent
else:
    from spike_events import MAX_REGULAR_DELTA, SpikeEvent

THRESHOLD = 0.05


def delta_encode(
    image: ArrayLike,
    threshold: float = THRESHOLD,
) -> list[list[SpikeEvent]]:
    """Encode each image row as an independent signed spike
        stream.

    Column indices are spike times and row streams are
        independent. Event
    input indices address the flattened image. Each threshold-
        sized intensity
    change emits one signed unit spike; additional units at the
        same pixel use
    delta time zero. Long gaps use time-only overflow events.
    """
    pixels = np.asarray(image, dtype=np.float64)
    if pixels.ndim == 1:
        pixels = pixels.reshape(1, -1)
    elif pixels.ndim != 2:
        raise ValueError("image must be a one- or two-dimensional
        array")
    if pixels.size == 0 or pixels.shape[0] == 0 or
        pixels.shape[1] == 0:
        raise ValueError("image must contain at least one pixel")
    if not np.all(np.isfinite(pixels)):
        raise ValueError("image must contain only finite pixel
        values")
    if np.any((pixels < 0.0) | (pixels > 1.0)):
        raise ValueError("image pixel values must be in [0, 1]")
    if not math.isfinite(threshold) or threshold <= 0.0:
        raise ValueError("threshold must be a finite positive
        number")

    row_streams: list[list[SpikeEvent]] = []
    width = pixels.shape[1]
    for row_index, row in enumerate(pixels):
        events: list[SpikeEvent] = []
        previous_value = 0.0
        last_event_time = 0

        for column_index, value in enumerate(row):
            change = float(value) - previous_value
            previous_value = float(value)
            amplitude = abs(change) / threshold
```

```
            spike_count = math.floor(amplitude + 1e-12)
            if spike_count == 0:
                continue

            polarity = 1 if change > 0.0 else -1
            delta_time = column_index - last_event_time
            while delta_time > MAX_REGULAR_DELTA:
                events.append(SpikeEvent(None, MAX_REGULAR_DELTA
        + 1, 0))
                last_event_time += MAX_REGULAR_DELTA + 1
                delta_time = column_index - last_event_time

            input_index = row_index * width + column_index
            events.append(SpikeEvent(input_index, delta_time,
        polarity))
            events.extend(
                SpikeEvent(input_index, 0, polarity)
                for _ in range(spike_count - 1)
            )
            last_event_time = column_index

        row_streams.append(events)

    return row_streams
```

Design notes: - The encoder emits **one event per pixel column** that has a non-zero change. A change of magnitude `Δx` greater than `threshold` produces `floor(|Δx|/τ)` spikes — the first at `delta_time` (the column offset from the last event) and the rest at `delta_time = 0` (simultaneous multiplicity). - **Overflow handling:** If the distance to the previous event exceeds 254 columns (the maximum representable delta), the encoder inserts multiple `SpikeEvent(None, 255, 0)` markers. Each marker advances the stream's accumulated absolute time by 255. - **Zero-delta multiplicity:** `events.extend(...)` adds repeated events at `delta_time = 0` for the remaining spike units. This preserves accuracy without consuming additional time slots.

---

15. `spike_events.py`

Purpose: Shared event representation for the software model. Defines the `SpikeEvent` namedtuple and the overflow-code constants.

Full source code:

```
"""Shared event representation for the software model."""

from typing import NamedTuple
```

```
OVERFLOW_CODE = 0xFF
MAX_REGULAR_DELTA = OVERFLOW_CODE - 1


class SpikeEvent(NamedTuple):
    """One signed unit spike, or a time-only overflow marker.

    Overflow events use input_index=None and polarity=0. Regular
        events use a
    non-negative input index and polarity -1 or +1.
    """

    input_index: int | None
    delta_time: int
    polarity: int

    @property
    def is_overflow(self) -> bool:
        return (
            self.input_index is None
            and self.delta_time == OVERFLOW_CODE
            and self.polarity == 0
        )
```

Key fields:

Field	Type	Purpose
`input_index`	`int | None`	Flattened pixel address for regular events; `None` for overflow markers
`delta_time`	`int`	Time offset since the previous event; `0xFF` (255) reserved as overflow
`polarity`	`int`	+1 or −1 for regular events; 0 for overflow markers

Design notes: - The `is_overflow` property is a single boolean check used by the decoder and reference model. - `OVERFLOW_CODE = 0xFF` is the same value used by the RTL to signal time advancement.

---

16. `snn_ref.py`

Purpose: Integer, event-driven reference model for the hidden and output LIF layers. This is the **golden model** against which RTL is compared.

Full source code:

```
"""Integer reference model for the project's differential-time
        SNN."""

from __future__ import annotations

from collections.abc import Iterable

import numpy as np

if __package__:
    from .spike_events import OVERFLOW_CODE, SpikeEvent
else:
    from spike_events import OVERFLOW_CODE, SpikeEvent

THETA = 16
BETA_SHIFT = 1


def lif_step(
    pot: int,
    dt: int,
    synaptic_input: int,
    theta: int = THETA,
    theta_low: int | None = None,
) -> tuple[int, int]:
    """Apply one signed event to a reset-to-mod LIF neuron.

    Returns the updated potential and a spike polarity (-1, 0, or
        +1).
    ``synaptic_input`` is the signed sum of the weights at this
        event time.
    """
    if dt < 0:
        raise ValueError("dt must be non-negative")
    if theta <= 0:
        raise ValueError("theta must be positive")
    if theta_low is not None and theta_low >= theta:
        raise ValueError("theta_low must be less than theta")

    decayed = int(pot) >> (int(dt) * BETA_SHIFT)
    summed = decayed + int(synaptic_input)
    if summed >= theta:
        return summed - theta, 1
```

```
    if theta_low is not None and summed <= theta_low:
        return summed - theta_low, -1
    return summed, 0


def run_image(
    streams: list[list[SpikeEvent]],
    hidden_weights: np.ndarray,
    output_weights: np.ndarray,
    theta: int = THETA,
    theta_low: int | None = None,
) -> tuple[int, np.ndarray]:
    """Run the integer reference model on a single image.

    Returns (digit, output_counts) where digit is the argmax of
        output spike
    counts and output_counts is a length-10 array of per-digit
        spike counts.

    Steps:
      1. Decode each row stream into absolute-time regular events.
      2. Repeatedly find the earliest pending input time.
      3. Sum all hidden-layer contributions at that time.
      4. Update every hidden LIF neuron.
      5. Collect hidden spikes into per-neuron time streams.
      6. Repeat earliest-time processing for the output layer.
      7. Count output spikes and return argmax.
    """
    if not streams:
        raise ValueError("streams must contain at least one input
        stream")
    if hidden_weights.ndim != 2:
        raise
        ValueError("hidden_weights must be a two-dimensional
        array")
    if output_weights.ndim != 2:
        raise
        ValueError("output_weights must be a two-dimensional
        array")
    if hidden_weights.shape[0] != 784:
        raise ValueError("hidden_weights must have 784 rows (one
        per input pixel)")
    if output_weights.shape[0] != 128:
        raise ValueError("output_weights must have 128 rows (one
        per hidden neuron)")
    if output_weights.shape[1] != 10:
        raise
        ValueError("output_weights must have 10 columns (one per
        digit)")

    input_streams = [
        _decode_input_stream(events, hidden_weights.shape[0],
        stream_index)
        for stream_index, events in enumerate(streams)
```

```
]
if not input_streams:
    raise ValueError("streams must contain at least one input
    stream")

hidden_count = hidden_weights.shape[1]
output_count = output_weights.shape[1]
hidden_potentials = [0] * hidden_count
hidden_spikes: list[list[tuple[int, int]]] = [[] for _ in
    range(hidden_count)]
input_positions = [0] * len(input_streams)
last_hidden_update_time = 0

while True:
    active_times = [
        events[input_positions[stream_index]][0]
        for stream_index, events in enumerate(input_streams)
        if input_positions[stream_index] < len(events)
    ]
    if not active_times:
        break

    event_time = min(active_times)
    synaptic_inputs = np.zeros(hidden_count, dtype=np.int64)
    for stream_index, events in enumerate(input_streams):
        position = input_positions[stream_index]
        if position < len(events) and events[position][0] ==
    event_time:
            _, input_index, polarity = events[position]
            synaptic_inputs += hidden_weights[input_index] *
    polarity
            input_positions[stream_index] += 1

    elapsed = event_time - last_hidden_update_time
    for neuron in range(hidden_count):
        potential, polarity = lif_step(
            hidden_potentials[neuron],
            elapsed,
            int(synaptic_inputs[neuron]),
            theta,
            theta_low,
        )
        hidden_potentials[neuron] = potential
        if polarity:
            hidden_spikes[neuron].append((event_time,
    polarity))
    last_hidden_update_time = event_time

output_potentials = [0] * output_count
output_counts = np.zeros(output_count, dtype=np.int64)
stream_positions = [0] * hidden_count
last_output_update_time = 0
```

```
    while True:
        active_times = [
            spikes[stream_positions[neuron]][0]
            for neuron, spikes in enumerate(hidden_spikes)
            if stream_positions[neuron] < len(spikes)
        ]
        if not active_times:
            break

        event_time = min(active_times)
        synaptic_inputs = np.zeros(output_count, dtype=np.int64)
        for neuron, spikes in enumerate(hidden_spikes):
            position = stream_positions[neuron]
            if position < len(spikes) and spikes[position][0] ==
        event_time:
                polarity = spikes[position][1]
                synaptic_inputs += output_weights[neuron] *
        polarity
                stream_positions[neuron] += 1

        elapsed = event_time - last_output_update_time
        for output in range(output_count):
            potential, polarity = lif_step(
                output_potentials[output],
                elapsed,
                int(synaptic_inputs[output]),
                theta,
                theta_low,
            )
            output_potentials[output] = potential
            if polarity:
                output_counts[output] += 1
        last_output_update_time = event_time

    return int(np.argmax(output_counts)), output_counts


if __name__ == "__main__":

        print("snn_ref.py: import lif_step() or run_image() to run
        the reference model")
```

Key functions and signals:

17. `quantize.py`

Purpose: Standalone float-to-signed-integer quantizer. Template — the deployment pipeline (checkpoint loading, layer-specific scales, post-quantization accuracy) is not yet complete.

Full source code:

```
"""Quantizes trained float weights to signed integers of W_W bits.

TEMPLATE: scale and rounding policy to be decided.
"""

import numpy as np

W_BITS = 4


def quantize_weights(w_float, bits=W_BITS):
    qmax = 2 ** (bits - 1) - 1
    scale = np.max(np.abs(w_float)) / qmax if
        np.max(np.abs(w_float)) > 0 else 1.0
    q = np.clip(np.round(w_float / scale), -qmax - 1,
        qmax).astype(np.int32)
    return q, scale
```

Key functions and signals:

Symbol	Purpose	Design notes
`quantize_weights(w_float, bits=4)`	Quantize a float weight matrix to signed integer	Applies global uniform scale per weight matrix; returns `(quantized, scale)`

Design notes: - The quantization is **uniform** (single global scale per weight matrix), not per-layer or per-row. The TODO comment explicitly says this is a template. - `W_BITS = 4` matches the RTL parameter `W_W = 4` (signed 4-bit weights). - The `clip` range `[-qmax-1, qmax]` allows one extra value beyond the signed range, which effectively saturates rather than wraps.

---

18. `export_mem.py`

Purpose: Generic writer for `$readmemh`-compatible hexadecimal files. Template — model-specific serialization (e.g., connecting events and weights to the correct memory addresses) is not yet implemented.

Full source code:

```
"""Writes $readmemh-compatible files for weights, streams and
        labels.

TEMPLATE: file formats must match the widths in rtl/util/
        pkg_snn.sv.
"""

import os


def write_hex(path, values, width_bits):
    """Write one value per line as zero-padded hex."""
    digits = (width_bits + 3) // 4
    os.makedirs(os.path.dirname(path), exist_ok=True)
    with open(path, "w") as f:
        for v in values:
            f.write(f"{int(v) & ((1 << width_bits) - 1):0{digits}
        x}")
```

Key functions and signals:

Symbol	Purpose	Design notes
`write_hex(path, values, width_bits)`	Write integer values to a hex file	One value per line, zero-padded to the specified bit width

Design notes: - The `digits` calculation rounds up to the nearest hex digit: 4 bits → 1 hex digit, 8 bits → 2 hex digits, 16 bits → 4 hex digits. - The mask `& ((1 << width_bits) - 1)` truncates to the specified bit width, discarding any higher bits. - The RTL `stream_rom.sv` and `weight_rom.sv` use `$readmemh` to load these files, so the format is zero-padded hex, one per line — exactly what `write_hex` produces.

---

19. `test_snn_model.py`

Purpose: Focused tests for differential-time event encoding and the integer reference model.

Full source code:

```
"""Focused tests for differential-time event encoding and the
        integer reference."""

import unittest

import numpy as np

from model.encode_mnist import delta_encode
```

```
from model.snn_ref import lif_step, run_image
from model.spike_events import SpikeEvent


class DeltaEncodeTests(unittest.TestCase):
    def test_emits_independent_signed_row_streams(self) -> None:
        image = [[0.0, 0.1, 0.2, 0.15], [0.0, 0.0, 0.05, 0.0]]
        streams = delta_encode(image)
        self.assertEqual(
            streams,
            [
                [
                    SpikeEvent(1, 1, 1),
                    SpikeEvent(1, 0, 1),
                    SpikeEvent(2, 1, 1),
                    SpikeEvent(2, 0, 1),
                    SpikeEvent(3, 1, -1),
                ],
                [SpikeEvent(6, 2, 1), SpikeEvent(7, 1, -1)],
            ],
        )

    def test_splits_long_gaps_with_time_only_overflows(self) ->
        None:
        image = np.zeros((1, 301))
        image[0, 300] = 1.0
        streams = delta_encode(image, threshold=0.5)
        self.assertEqual(
            streams,
            [
                [
                    SpikeEvent(None, 255, 0),
                    SpikeEvent(300, 45, 1),
                    SpikeEvent(300, 0, 1),
                ]
            ],
        )

    def test_rejects_invalid_images_and_thresholds(self) -> None:
        for image in ([], [0.0, np.nan], [0.0, 1.1]):
            with self.subTest(image=image):
                with self.assertRaises(ValueError):
                    delta_encode(image)
        with self.assertRaises(ValueError):
            delta_encode([0.0], threshold=0.0)


class LifStepTests(unittest.TestCase):
    def test_decay_and_positive_reset_to_mod(self) -> None:
        self.assertEqual(lif_step(8, 1, 14), (2, 1))

    def test_negative_threshold_and_signed_spike(self) -> None:
```

```
        self.assertEqual(lif_step(-8, 1, -14, theta_low=-16),
        (-2, -1))

    def test_applies_only_one_threshold_reset_per_event(self) ->
        None:
        self.assertEqual(lif_step(0, 0, 35), (19, 1))


class RunImageTests(unittest.TestCase):
    def test_merges_simultaneous_hidden_spikes_for_output_layer(self)
        -> None:
        streams = [
            [SpikeEvent(0, 0, 1)],
            [SpikeEvent(1, 0, 1)],
        ]
        w_hidden = np.array([[10, 10], [10, 10]], dtype=np.int8)
        w_out = np.array([[9, 7], [9, 7]], dtype=np.int8)

        digit, counts = run_image(streams, w_hidden, w_out)

        self.assertEqual(digit, 0)
        np.testing.assert_array_equal(counts, [1, 0])

    def test_zero_delta_spikes_from_one_row_are_processed_separately(self)
        -> None:
        streams = [[SpikeEvent(0, 0, 1), SpikeEvent(0, 0, 1)]]
        w_hidden = np.array([[10]], dtype=np.int8)
        w_out = np.array([[16]], dtype=np.int8)

        _, counts = run_image(streams, w_hidden, w_out)

        np.testing.assert_array_equal(counts, [1])

    def test_overflow_advances_time_without_updating_neurons(self)
        -> None:
        streams = [
            [
                SpikeEvent(0, 0, 1),
                SpikeEvent(None, 255, 0),
                SpikeEvent(1, 0, 1),
            ]
        ]
        w_hidden = np.array([[8], [16]], dtype=np.int8)
        w_out = np.array([[22]], dtype=np.int8)

        _, counts = run_image(streams, w_hidden, w_out, theta=22)

        np.testing.assert_array_equal(counts, [0])

    def test_rejects_malformed_events_and_weight_shapes(self) ->
        None:
        with self.assertRaises(ValueError):
            run_image(
```

```
                [[SpikeEvent(None, 255, 1)]],
                np.ones((1, 1), dtype=np.int8),
                np.ones((1, 1), dtype=np.int8),
            )
        with self.assertRaises(ValueError):
            run_image(
                [],
                np.ones((1, 2), dtype=np.int8),
                np.ones((1, 1), dtype=np.int8),
            )


if __name__ == "__main__":
    unittest.main()
```

20. `test_train_snn.py`

Purpose: Tests for MNIST IDX loading and differential-time batch input preparation.

Full source code:

```
"""Tests for MNIST IDX loading and differential-time training
        input preparation."""

import struct
import tempfile
import unittest
from pathlib import Path

import numpy as np

from model.train_snn import (
    encode_batch,
    load_mnist,
    read_idx_images,
    read_idx_labels,
)


class IdxReaderTests(unittest.TestCase):
    def test_reads_mnist_image_and_label_files(self) -> None:
        images = np.arange(2 * 28 * 28,
        dtype=np.uint8).reshape(2, 28, 28)
        labels = np.array([3, 7], dtype=np.uint8)
        with tempfile.TemporaryDirectory() as directory:
            root = Path(directory)
            image_path = root / "images"
            label_path = root / "labels"
            image_path.write_bytes(
```

```
                struct.pack(">IIII", 2051, 2, 28, 28) +
            images.tobytes()
                )
                label_path.write_bytes(struct.pack(">II", 2049, 2) +
            labels.tobytes())


            np.testing.assert_array_equal(read_idx_images(image_path),
            images)

            np.testing.assert_array_equal(read_idx_labels(label_path),
            labels)

        def test_rejects_bad_idx_headers_and_sizes(self) -> None:
            with tempfile.TemporaryDirectory() as directory:
                image_path = Path(directory) / "images"
                image_path.write_bytes(struct.pack(">IIII", 0, 1, 28,
            28))
                with self.assertRaises(ValueError):
                    read_idx_images(image_path)

                label_path = Path(directory) / "labels"
                label_path.write_bytes(struct.pack(">II", 2049, 2) +
            b"\x01")
                with self.assertRaises(ValueError):
                    read_idx_labels(label_path)

        def test_loads_supplied_mnist_dataset(self) -> None:
            root = Path(__file__).resolve().parents[1] / "data" /
            "mnist"
            train_images, train_labels, test_images, test_labels =
            load_mnist(root)
            self.assertEqual(train_images.shape, (60000, 28, 28))
            self.assertEqual(train_labels.shape, (60000,))
            self.assertEqual(test_images.shape, (10000, 28, 28))
            self.assertEqual(test_labels.shape, (10000,))


    class DifferentialInputTests(unittest.TestCase):
        def test_builds_parallel_signed_row_events(self) -> None:
            images = np.zeros((1, 28, 28), dtype=np.float32)
            images[0, 0, 0] = 0.1
            images[0, 0, 1] = 0.2
            images[0, 0, 2:] = 0.2
            images[0, 1, 0] = 0.05
            images[0, 1, 1] = 0.0

            sequence = encode_batch(images)

            self.assertEqual(sequence.shape, (28, 1, 784))
            self.assertEqual(sequence[0, 0, 0], 2.0)
            self.assertEqual(sequence[1, 0, 1], 2.0)
            self.assertEqual(sequence[0, 0, 28], 1.0)
```

```
        self.assertEqual(sequence[1, 0, 29], -1.0)
        self.assertEqual(np.count_nonzero(sequence[2:]), 0)

    def test_rejects_bad_batch_and_threshold(self) -> None:
        with self.assertRaises(ValueError):
            encode_batch(np.zeros((1, 27, 28), dtype=np.float32))
        with self.assertRaises(ValueError):
            encode_batch(np.zeros((1, 28, 28), dtype=np.float32),
        threshold=0.0)


if __name__ == "__main__":
    unittest.main()
```

21. `requirements-model.txt`

Purpose: Python dependencies for CPU-capable training. The training environment needs:

```
--extra-index-url https://download.pytorch.org/whl/cpu
numpy>=1.24
snntorch>=0.9.4
torch>=2.0
```

- `numpy>=1.24` — required for the arrays in `encode_batch` and the reference model.
- `snntorch>=0.9.4` — the snnTorch framework providing `Leaky`, `surrogate`, and `functional`.
- `torch>=2.0` — the PyTorch deep-learning framework.

The `--extra-index-url` points to the PyTorch CPU-only wheel repository to avoid needing a CUDA-capable GPU.

Part VII — RTL Architecture

22. `pkg_snn.sv`

Purpose: Central parameter source for the SNN accelerator. All network sizes and numeric widths are defined here, and every other RTL file should import from it — changing sizes in one place updates the entire design.

Key parameters:

Parameter	Value	Meaning
IMAGE_ROWS	28	Number of input streams (one per image row)
IMAGE_COLS	28	Logical image columns
N_INPUT	784	Flattened input address count
N_INPUT_STREAMS	28	Number of independent row event streams
N_HIDDEN	128	Number of hidden LIF neurons (time-multiplexed)
N_OUTPUT	10	Number of output neurons (digits 0–9)
W_W	4	Signed synaptic weight width
POT_W	16	Signed membrane potential width
TIME_W	16	Absolute time register width
DT_W	8	Delta-time field in spike events
THETA	16	Integer firing threshold
OVERFLOW_CODE	8'hFF	Reserved all-ones delta-time value

Full source code:

```
// pkg_snn.sv
// Central parameters for the SNN accelerator. Change sizes and
        widths here only.
package pkg_snn;

  // ---- Network size ----
  localparam int IMAGE_ROWS = 28;
  localparam int IMAGE_COLS = 28;
  localparam int N_INPUT   = IMAGE_ROWS * IMAGE_COLS; // input
        pixel weights
  localparam int N_INPUT_STREAMS =
        IMAGE_ROWS; // one differential stream per row
  localparam int N_HIDDEN  = 128;   // hidden LIF neurons (time-
        multiplexed)
  localparam int N_OUTPUT  = 10;    // one per digit

  // ---- Bit widths (size from Python model's observed maxima)
        ----
  localparam int W_W       = 4;     // synaptic weight width
        (signed)
  localparam int POT_W     = 16;    // membrane potential width
        (signed)
  localparam int TIME_W    = 16;    // absolute time register
        width
  localparam int DT_W      = 8;     // delta-time field in spike
        events
```

```
  // ---- Neuron parameters ----
  localparam int THETA     = 16;    // firing threshold (integer,
        same units as weights)

  // ---- Stream ----
  localparam int OVERFLOW_CODE =
        8'hFF; // reserved all-ones delta = timer overflow spike

endpackage
```

Design notes: - All parameters are `localparam` (compile-time constants), so the synthesis tool can inline them and optimize accordingly. - The bit widths (`W_W`, `POT_W`, `TIME_W`, `DT_W`) are derived from Python-model observations: the 4-bit weights match the `quantize.py` template, and the 16-bit potential/time registers give headroom for the 16-bit absolute time and 16-bit potential values.

---

23. `event_sched.sv`

Purpose: Serial minimum-time search across pending row-stream events. Produces the earliest absolute event time and a mask indicating which streams are tied at that time.

Key signals:

Signal	Direction	Purpose
`scan_req`	Input	Pulse while idle to trigger a new minimum scan
`ev_ready`	Input	Consumer accepts the held event; returns the state to IDLE
`next_t [N]`	Input	Array of per-stream current absolute times
`valid [N]`	Input	Whether each stream has a pending event
`negative [N]`	Input	1 for negative-spike events
`ev_valid`	Output	Pulses high when an event is available
`ev_time`	Output	

Signal	Direction	Purpose
		The earliest absolute time among all streams
`ev_mask`	Output	Bit-N set if stream N tied at the minimum time
`ev_negative`	Output	1 for negative events among the tied streams
`scan_busy`	Output	High while in any non-IDLE state

State machine:

State	Purpose
`IDLE`	Waiting for `scan_req` pulse; resets all outputs
`SCAN`	Checks one stream per clock, maintaining the running minimum; after `N` clocks, transitions to `FINALIZE`
`FINALIZE`	Pulses `scan_done`, captures the winning time and mask, holds the event valid until `ev_ready`
`HOLD`	Holds `ev_valid`, `ev_time`, `ev_mask` stable until the consumer signals `ev_ready`

Full source code:

```
// Serial minimum-time search for one layer's pending input
        events.
import pkg_snn::*;

module event_sched #(
    parameter int N = N_INPUT_STREAMS
) (
    input  logic                    clk,
    input  logic                    rst_n,
    input  logic                    scan_req,    // pulse while
        idle to scan once
    input  logic                    ev_ready,    // accept a held
        event
```

```
    input  logic [TIME_W-1:0]       next_t [N],   // hold stable
        until scan_done
    input  logic                    valid [N],
    input  logic                    negative [N],// 1 denotes a
        negative spike
    output logic                    scan_busy,
    output logic                    scan_done,   // pulse,
        including an empty scan
    output logic                    ev_valid,    // held until
        ev_ready
    output logic [TIME_W-1:0]       ev_time,
    output logic [N-1:0]            ev_mask,
    output logic [N-1:0]            ev_negative
);

  localparam int INDEX_W = (N > 1) ? $clog2(N) : 1;

  typedef enum logic [1:0] {
    IDLE,
    SCAN,
    FINALIZE,
    HOLD
  } state_e;

  state_e state;
  logic [INDEX_W-1:0] scan_index;
  logic [TIME_W-1:0] min_time_r;
  logic [N-1:0] mask_r;
  logic [N-1:0] negative_r;
  logic found_r;

  assign scan_busy = (state != IDLE);

  always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
      state       <= IDLE;
      scan_index  <= '0;
      min_time_r  <= '0;
      mask_r      <= '0;
      negative_r  <= '0;
      found_r     <= 1'b0;
      scan_done   <= 1'b0;
      ev_valid    <= 1'b0;
      ev_time     <= '0;
      ev_mask     <= '0;
      ev_negative <= '0;
    end else begin
      scan_done <= 1'b0;
      case (state)
        IDLE: begin
          if (scan_req) begin
            scan_index <= '0;
```

```
    min_time_r <= '0;
    mask_r     <= '0;
    negative_r <= '0;
    found_r    <= 1'b0;
    ev_valid   <= 1'b0;
    ev_time    <= '0;
    ev_mask    <= '0;
    ev_negative <= '0;
    state      <= SCAN;
  end
end

SCAN: begin
  if (valid[scan_index]) begin
    if (!found_r || (next_t[scan_index] < min_time_r))
begin
      min_time_r <= next_t[scan_index];
      mask_r     <= {{(N-1){1'b0}}, 1'b1} << scan_index;
      negative_r <= {{(N-1){1'b0}}, negative[scan_index]}
<< scan_index;
      found_r    <= 1'b1;
    end else if (next_t[scan_index] == min_time_r) begin
      mask_r[scan_index]     <= 1'b1;
      negative_r[scan_index] <= negative[scan_index];
    end
  end

  if (scan_index == INDEX_W'(N - 1)) begin
    state <= FINALIZE;
  end else begin
    scan_index <= scan_index + 1'b1;
  end
end

FINALIZE: begin
  scan_done <= 1'b1;
  if (found_r) begin
    ev_valid    <= 1'b1;
    ev_time     <= min_time_r;
    ev_mask     <= mask_r;
    ev_negative <= negative_r;
    state       <= HOLD;
  end else begin
    ev_valid    <= 1'b0;
    ev_time     <= '0;
    ev_mask     <= '0;
    ev_negative <= '0;
    state       <= IDLE;
  end
end

HOLD: begin
```

```
          if (ev_ready) begin
            ev_valid    <= 1'b0;
            ev_time     <= '0;
            ev_mask     <= '0;
            ev_negative <= '0;
            state       <= IDLE;
          end
        end

        default: begin
          state       <= IDLE;
          ev_valid    <= 1'b0;
          ev_time     <= '0;
          ev_mask     <= '0;
          ev_negative <= '0;
        end
      endcase
    end
  end
endmodule
```

Design notes: - The scheduler is a **serial minimum search**: it checks one stream per clock, maintaining the running minimum. For 28 streams, scanning takes 28 clock cycles. - **Tie handling:** When multiple streams share the same minimum time, a “mask” is created that marks all tied streams. The consumer must handle all tied events simultaneously (or the top-level may need to arbitrate). - The scheduler does **not** consume stream memory itself; the top-level must advance stream state and addresses based on the accepted event.

---

24. `time_integrator.sv`

Purpose: Converts a single input’s differential (delta) time stream into absolute time. Overflow spikes advance time without generating a spike.

Key signals:

Signal	Direction	Purpose
`step`	Input	Pulse to consume the next delta word from the stream
`delta`	Input	The 8-bit delta-time value from the stream

Signal	Direction	Purpose
`abs_time`	Output	Accumulated absolute time in 16 bits
`is_overflow`	Output	High when `delta == 8'hFF` (time-only marker)

Full source code:

```
// time_integrator.sv
// Converts one input's differential (delta) time stream into
        absolute time.
// Overflow spikes advance time without generating a spike.
// TEMPLATE: interface and basic accumulation only.
import pkg_snn::*;

module time_integrator (
    input  logic                clk,
    input  logic                rst_n,
    input  logic                clear,
    input  logic                step,          // consume next
        delta from stream
    input  logic [DT_W-1:0]     delta,         // delta time from
        stream
    output logic [TIME_W-1:0]   abs_time,      // accumulated
        absolute time
    output logic                is_overflow    // delta ==
        OVERFLOW_CODE
);

  logic [TIME_W-1:0] acc;

  assign is_overflow = (delta == DT_W'(OVERFLOW_CODE));

  always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n)       acc <= '0;
    else if (clear)   acc <= '0;
    else if (step)    acc <= acc + TIME_W'(delta);
  end

  assign abs_time = acc;

endmodule
```

Design notes: - The module accumulates delta times additively: `acc = acc + delta`. For a 16-bit time register and initial absolute time of 0, this can represent up to 65535 time units. - The `is_overflow` output is exposed so the caller can distinguish time-only markers from event

spikes. - The simple accumulator is a **template** — end-of-stream handling and full integration with the top-level stream banks are unfinished.

---

25. `lif_neuron.sv`

Purpose: Single signed-event LIF neuron with reset-to-mod thresholding. Instantiated both in the time-multiplexed hidden layer and as the parallel building block for the output layer.

Key signals:

Signal	Direction	Purpose
`ev_valid`	Input	Event present (delta and synaptic input are valid)
`dt`	Input	8-bit elapsed time since the previous event
`synaptic_input`	Input	Signed weighted sum of the event time
`spike_valid`	Output	High when the neuron fires a positive spike
`spike_negative`	Output	High when the neuron fires a negative spike (when `ENABLE_NEGATIVE_SPIKES=1`)
`pot`	Output	Current membrane potential

Full source code:

```
// Single signed-event LIF neuron with reset-to-mod thresholding.
import pkg_snn::*;

module lif_neuron #(
    parameter bit ENABLE_NEGATIVE_SPIKES = 1'b0,
    parameter int THETA_LOW = -THETA
) (
    input  logic                    clk,
    input  logic                    rst_n,
    input  logic                    clear,
    input  logic                    ev_valid,
    input  logic [DT_W-1:0]         dt,
    input  logic signed [POT_W-1:0] synaptic_input,
    output logic                    spike_valid,
```

```
    output logic                      spike_negative,
    output logic signed [POT_W-1:0]   pot
);

  localparam logic signed [POT_W-1:0] THETA_VALUE = POT_W'(THETA);
  localparam logic signed [POT_W-1:0] THETA_LOW_VALUE =
        POT_W'(THETA_LOW);

  logic signed [POT_W-1:0] pot_r;
  logic signed [POT_W-1:0] decayed;
  logic signed [POT_W-1:0] summed;
  logic                    fire_positive;
  logic                    fire_negative;

  always_comb begin
    decayed       = pot_r >>> dt;
    summed        = decayed + synaptic_input;
    fire_positive = ev_valid && (summed >= THETA_VALUE);
    fire_negative = ev_valid && ENABLE_NEGATIVE_SPIKES
                    && (summed <= THETA_LOW_VALUE);
  end

  always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
      pot_r         <= '0;
      spike_valid   <= 1'b0;
      spike_negative <= 1'b0;
    end else if (clear) begin
      pot_r         <= '0;
      spike_valid   <= 1'b0;
      spike_negative <= 1'b0;
    end else begin
      spike_valid    <= fire_positive || fire_negative;
      spike_negative <= fire_negative;
      if (ev_valid) begin
        if (fire_positive)
          pot_r <= summed - THETA_VALUE;
        else if (fire_negative)
          pot_r <= summed - THETA_LOW_VALUE;
        else
          pot_r <= summed;
      end
    end
  end

  assign pot = pot_r;

endmodule
```

Design notes: - **Decay:** `pot_r >>> dt` — right shift by `dt` bits implements the leak. With `dt=0`, no decay occurs. - **Threshold:** `summed >= THETA_VALUE` fires a positive spike. After firing, `pot_r = summed - THETA_VALUE` (reset-to-mod). - **Optional negative threshold:** When `ENABLE_NEGATIVE_SPIKES=1`, `summed <= THETA_LOW_VALUE` fires a negative spike and `pot_r = summed - THETA_LOW_VALUE`. - **Registering:** The outputs `spike_valid` and `spike_negative` are registered on the clock edge, giving a one-cycle delay. The `clear` signal overrides everything, resetting the neuron to its idle state.

---

26. `lif_layer.sv`

Purpose: Time-multiplexed (single datapath) hidden LIF layer. Stores 128 membrane potentials but reuses one arithmetic datapath — processing one neuron per clock cycle for up to 128 cycles per accepted event.

Key signals:

Signal	Direction	Purpose
`ev_valid`	Input	Event present for this layer
`dt`	Input	8-bit delta time
`ev_negative`	Input	1 for negative-polarity events
`w_row [0:N-1]`	Input	Weight row: 128 packed 4-bit signed weights
`busy`	Output	High while the layer processes a neuron
`event_done`	Output	Pulses when all 128 neurons have been processed
`spk_valid [0:N-1]`	Output	Spike from each of the 128 hidden neurons
`spk_negative [0:N-1]`	Output	Negative spikes from each neuron
`pot [0:N-1]`	Output	

Signal	Direction	Purpose
		Membrane potential of each of the 128 neurons

Full source code:

```
// lif_layer.sv
// Time-multiplexed LIF layer sharing one signed event datapath.
// TEMPLATE: currently instantiates N_HIDDEN neurons in parallel
        for simplicity.
// Replace with a time-multiplexed scheme (a few lanes + state
        BRAM) once the
// resource estimate says it is needed.
import pkg_snn::*;

module lif_layer #(
    parameter int N = N_HIDDEN,
    parameter bit ENABLE_NEGATIVE_SPIKES = 1'b0,
    parameter int THETA_LOW = -THETA
) (
    input  logic                     clk,
    input  logic                     rst_n,
    input  logic                     clear,
    input  logic                     ev_valid,
    input  logic [DT_W-1:0]          dt,
    input  logic                     ev_negative,
    input  logic signed [W_W-1:0]    w_row [0:N-1],   // weight
        per neuron for this input
    output logic                     busy,
    output logic                     event_done,
    output logic                     spk_valid [0:N-1],
    output logic                     spk_negative [0:N-1],
    output logic signed [POT_W-1:0]  pot       [0:N-1]
);

  localparam int INDEX_W = (N > 1) ? $clog2(N) : 1;
  localparam logic signed [POT_W-1:0] THETA_VALUE = POT_W'(THETA);
  localparam logic signed [POT_W-1:0] THETA_LOW_VALUE =
        POT_W'(THETA_LOW);

  logic signed [POT_W-1:0] pot_r [0:N-1];
  logic [INDEX_W-1:0] neuron_index;
  logic busy_r;
  logic [DT_W-1:0] dt_r;
  logic ev_negative_r;
  logic signed [POT_W-1:0] weight_extended;
  logic signed [POT_W-1:0] signed_weight;
  logic signed [POT_W-1:0] decayed;
  logic signed [POT_W-1:0] summed;
  logic fire_positive;
```

```
logic fire_negative;

assign busy = busy_r;
assign weight_extended = POT_W'(w_row[neuron_index]);
assign signed_weight = ev_negative_r ? -weight_extended :
      weight_extended;
assign decayed = pot_r[neuron_index] >>> dt_r;
assign summed = decayed + signed_weight;
assign fire_positive = (summed >= THETA_VALUE);
assign fire_negative = ENABLE_NEGATIVE_SPIKES && (summed <=
      THETA_LOW_VALUE);

generate
  for (genvar i = 0; i < N; i++) begin : g_outputs
    assign pot[i] = pot_r[i];
  end
endgenerate

always_ff @(posedge clk or negedge rst_n) begin
  if (!rst_n) begin
    neuron_index <= '0;
    busy_r       <= 1'b0;
    dt_r         <= '0;
    ev_negative_r <= 1'b0;
    event_done   <= 1'b0;
    for (int i = 0; i < N; i++) begin
      pot_r[i]       <= '0;
      spk_valid[i]   <= 1'b0;
      spk_negative[i] <= 1'b0;
    end
  end else if (clear) begin
    neuron_index <= '0;
    busy_r       <= 1'b0;
    dt_r         <= '0;
    ev_negative_r <= 1'b0;
    event_done   <= 1'b0;
    for (int i = 0; i < N; i++) begin
      pot_r[i]       <= '0;
      spk_valid[i]   <= 1'b0;
      spk_negative[i] <= 1'b0;
    end
  end else begin
    event_done <= 1'b0;
    for (int i = 0; i < N; i++) begin
      spk_valid[i]    <= 1'b0;
      spk_negative[i] <= 1'b0;
    end

    if (!busy && !event_done && ev_valid) begin
      dt_r          <= dt;
      ev_negative_r <= ev_negative;
      neuron_index  <= '0;
```

```
        busy_r        <= 1'b1;
      end else if (busy) begin
        spk_valid[neuron_index]    <= fire_positive ||
        fire_negative;
        spk_negative[neuron_index] <= fire_negative;
        if (fire_positive)
          pot_r[neuron_index] <= summed - THETA_VALUE;
        else if (fire_negative)
          pot_r[neuron_index] <= summed - THETA_LOW_VALUE;
        else
          pot_r[neuron_index] <= summed;

        if (neuron_index == INDEX_W'(N - 1)) begin
          neuron_index <= '0;
          busy_r       <= 1'b0;
          event_done   <= 1'b1;
        end else begin
          neuron_index <= neuron_index + 1'b1;
        end
      end
    end
  end
endmodule
```

Design notes: - The module has a **state machine** with two states: `IDLE` (waiting for `ev_valid`) and `BUSY` (processing neurons sequentially). - On the first cycle, the event is latched (`dt_r`, `ev_negative_r`) and `busy_r` is set to high. - Each subsequent clock cycle processes one neuron: `neuron_index` increments from 0 to 127, the neuron's potential is decayed, added to the signed weight, compared against the threshold, and the result is stored. - `event_done` pulses when all 128 neurons have been processed; at that point, `busy_r` is cleared. - The module is currently **parallel for simplicity** (line 3: "TEMPLATE: currently instantiates N_HIDDEN neurons in parallel for simplicity"). The comment suggests the time-multiplexed scheme may not be fully implemented in code yet. This is a known gap.

---

27. `lif_layer_parallel.sv`

Purpose: Parallel LIF layer for the small (10-neuron) output layer. Each neuron gets its own dedicated datapath.

Full source code:

```
// Parallel LIF layer used for the small output layer.
import pkg_snn::*;

module lif_layer_parallel #(
```

```
    parameter int N = N_OUTPUT,
    parameter bit ENABLE_NEGATIVE_SPIKES = 1'b0,
    parameter int THETA_LOW = -THETA
) (
    input  logic                     clk,
    input  logic                     rst_n,
    input  logic                     clear,
    input  logic                     ev_valid,
    input  logic [DT_W-1:0]          dt,
    input  logic                     ev_negative,
    input  logic signed [W_W-1:0]    w_row [0:N-1],
    output logic                     spk_valid [0:N-1],
    output logic                     spk_negative [0:N-1],
    output logic signed [POT_W-1:0]  pot [0:N-1]
);

  generate
    for (genvar i = 0; i < N; i++) begin : g_neuron
      logic signed [POT_W-1:0] weight_extended;
      logic signed [POT_W-1:0] signed_weight;

      assign weight_extended = POT_W'(w_row[i]);
      assign signed_weight = ev_negative ? -weight_extended :
        weight_extended;

      lif_neuron #(
          .ENABLE_NEGATIVE_SPIKES (ENABLE_NEGATIVE_SPIKES),
          .THETA_LOW              (THETA_LOW)
      ) u_neuron (
          .clk             (clk),
          .rst_n           (rst_n),
          .clear           (clear),
          .ev_valid        (ev_valid),
          .dt              (dt),
          .synaptic_input  (signed_weight),
          .spike_valid     (spk_valid[i]),
          .spike_negative  (spk_negative[i]),
          .pot             (pot[i])
      );
    end
  endgenerate

endmodule
```

Design notes: - The generate block instantiates 10 `lif_neuron` modules, one per output neuron. Each neuron receives its own weight from the weight row. - The module is fully **combinational in datapath** — all 10 neurons update on the same clock cycle. This is much simpler than the time-multiplexed hidden layer. - The output layer is a **structural**

parallel stage; the design notes warn to treat it carefully during equivalence work because hidden spike timing and simultaneous spike aggregation determine its input events.

---

28. `spike_counter.sv` + `argmax.sv`

Purpose: Count output spikes and select the winning digit.

Full source code — `spike_counter.sv`:

```
// spike_counter.sv
// Counts output spikes per output neuron for the current image.
import pkg_snn::*;

module spike_counter (
    input  logic                    clk,
    input  logic                    rst_n,
    input  logic                    clear,
    input  logic [N_OUTPUT-1:0]     spk_in,       // one spike
        pulse per output neuron
    output logic [15:0]             count [N_OUTPUT]
);

  always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
      for (int k = 0; k < N_OUTPUT; k++) count[k] <= '0;
    end else if (clear) begin
      for (int k = 0; k < N_OUTPUT; k++) count[k] <= '0;
    end else begin
      for (int k = 0; k < N_OUTPUT; k++)
        if (spk_in[k]) count[k] <= count[k] + 16'd1;
    end
  end

endmodule
```

Full source code — `argmax.sv`:

```
// argmax.sv
// Selects the output neuron with the highest spike count.
// Ties resolve to the lowest index.
import pkg_snn::*;

module argmax (
    input  logic [15:0]             count [N_OUTPUT],
    output logic [3:0]              digit
);

  always_comb begin
    digit = '0;
```

```
    for (int k = 1; k < N_OUTPUT; k++)
      if (count[k] > count[digit]) digit = 4'(k);
  end

endmodule
```

Design notes: - The counter increments each digit's count by one for every spike pulse. - The `argmax` module compares counts using `>` (not `>=`), so ties resolve to the lower digit index — a standard convention. - The counter is cleared by `clear_state` at the start of each run, and the argmax result is meaningful only after the run completes.

---

29. `stream_rom.sv` + `weight_rom.sv`

Purpose: Load stream and weight data from hexadecimal files using `$readmemh`.

Full source code — `stream_rom.sv`:

```
// stream_rom.sv
// Encoded spike streams (delta-time events) and labels for test
        images.
// Initialized from files written by model/export_mem.py.
import pkg_snn::*;

module stream_rom #(
    parameter string INIT_FILE  = "data/streams/stream.mem",
    parameter int    DEPTH      = 65536
) (
    input  logic                clk,
    input  logic [15:0]         addr,
    output logic [DT_W-1:0]     data
);

  logic [DT_W-1:0] mem [DEPTH];

  initial $readmemh(INIT_FILE, mem);

  always_ff @(posedge clk) data <= mem[addr];

endmodule
```

Full source code — `weight_rom.sv`:

```
// weight_rom.sv
// Hidden-layer weights, one row per input pixel, one column per
        hidden neuron.
// Initialized from a hex file exported by model/export_mem.py.
import pkg_snn::*;
```

```
module weight_rom #(
    parameter string INIT_FILE = "data/weights/hidden_w.mem",
    parameter int    N_NEURONS = N_HIDDEN,
    parameter int    DEPTH     = N_INPUT,
    parameter int    ADDR_W    = 10
) (
    input  logic                    clk,
    input  logic [ADDR_W-1:0]       addr,
    output logic [N_NEURONS*W_W-1:0] data
);

  logic [N_NEURONS*W_W-1:0] mem [DEPTH];

  initial $readmemh(INIT_FILE, mem);

  always_ff @(posedge clk) data <= mem[addr];

endmodule
```

Design notes: - Both ROMs use `$readmemh` in an `initial` block to load the hex files. Because the files are **git-ignored**, the RTL will elaborate but fail at simulation if the files are missing. - The ROM files are intended to be generated by `model/export_mem.py` (not yet implemented), and the format is not yet fully defined (see later section). - `stream_rom` reads `DT_W = 8` bits per address; `weight_rom` reads `N_NEURONS * W_W = 128 * 4 = 512` bits per address.

---

30. `snn_top.sv`

Purpose: Top-level event-driven SNN datapath. Structures the event path and provides the low-level control interface.

Key ports:

Port	Direction	Purpose
`clk`	Input	FPGA system clock
`rst_n`	Input	Active-low asynchronous reset
`start`	Input	Starts one bounded inference run
`done`	Output	One-cycle completion pulse
`digit [3:0]`	Output	Winner digit (0–9)

Key internal blocks:

Block	Purpose
`stream_rom u_stream`	Reads `data/streams/stream.mem` for the current delta-time word
`weight_rom u_hidden_weights`	Reads `data/weights/hidden_w.mem` (128×4 bits per row)
`weight_rom u_output_weights`	Reads `data/weights/output_w.mem` (10×4 bits per row)
`event_sched u_input_scheduler`	4-state minimum search across the 28 streams
`lif_layer u_hidden_layer`	Time-multiplexed hidden layer
`lif_layer_parallel u_output_layer`	Parallel 10-neuron output layer
`spike_counter u_output_counter`	Counts spikes per digit
`argmax u_argmax`	Selects the winning digit

Full source code:

```
// Top-level event-driven SNN datapath.
// The control window is deliberately bounded while the stream end
        protocol is
// finalized; all accelerator layers are nevertheless instantiated
        and clocked.
import pkg_snn::*;

module snn_top (
    input  logic        clk,
    input  logic        rst_n,
    input  logic        start,
    output logic        done,
    output logic [3:0]  digit
);

  localparam int RUN_CYCLES = 1024;
  localparam int RUN_W = $clog2(RUN_CYCLES + 1);

  logic clear_state;
  logic running;
  logic [RUN_W-1:0] run_count;
  logic [15:0] stream_addr;
  logic [DT_W-1:0] stream_data;
  logic [TIME_W-1:0] next_time [0:N_INPUT_STREAMS-1];
  logic stream_valid [0:N_INPUT_STREAMS-1];
  logic stream_negative [0:N_INPUT_STREAMS-1];

  logic scan_busy;
  logic scan_done;
  logic hidden_busy;
```

```
logic hidden_event_done;
logic ev_valid;
logic [TIME_W-1:0] ev_time;
logic [N_INPUT_STREAMS-1:0] ev_mask;
logic [N_INPUT_STREAMS-1:0] ev_negative;

logic [N_HIDDEN*W_W-1:0] hidden_weight_data;
logic signed [W_W-1:0] hidden_w_row [0:N_HIDDEN-1];
logic hidden_spike [0:N_HIDDEN-1];
logic hidden_negative [0:N_HIDDEN-1];
logic signed [POT_W-1:0] hidden_pot [0:N_HIDDEN-1];
logic hidden_any_spike;
logic hidden_any_negative;

logic [N_OUTPUT*W_W-1:0] output_weight_data;
logic signed [W_W-1:0] output_w_row [0:N_OUTPUT-1];
logic output_spike [0:N_OUTPUT-1];
logic output_negative [0:N_OUTPUT-1];
logic signed [POT_W-1:0] output_pot [0:N_OUTPUT-1];
logic [N_OUTPUT-1:0] output_spike_bus;
logic [15:0] output_count [0:N_OUTPUT-1];

stream_rom #(
    .INIT_FILE ("data/streams/stream.mem")
) u_stream (
    .clk  (clk),
    .addr (stream_addr),
    .data (stream_data)
);

genvar s;
generate
  for (s = 0; s < N_INPUT_STREAMS; s++) begin : g_stream
    assign next_time[s] = TIME_W'(stream_data);
    assign stream_valid[s] = running;
    assign stream_negative[s] = 1'b0;
  end
endgenerate

always_comb begin
  hidden_any_spike = 1'b0;
  hidden_any_negative = 1'b0;
  for (int h = 0; h < N_HIDDEN; h++) begin
    hidden_any_spike = hidden_any_spike || hidden_spike[h];
    hidden_any_negative = hidden_any_negative ||
      hidden_negative[h];
  end
end

weight_rom #(
    .INIT_FILE  ("data/weights/hidden_w.mem"),
    .N_NEURONS  (N_HIDDEN),
```

```
    .DEPTH      (N_INPUT),
    .ADDR_W     (10)
) u_hidden_weights (
    .clk  (clk),
    .addr (stream_addr[9:0]),
    .data (hidden_weight_data)
);

weight_rom #(
    .INIT_FILE  ("data/weights/output_w.mem"),
    .N_NEURONS  (N_OUTPUT),
    .DEPTH      (N_HIDDEN),
    .ADDR_W     (7)
) u_output_weights (
    .clk  (clk),
    .addr (stream_addr[6:0]),
    .data (output_weight_data)
);

generate
  for (s = 0; s < N_HIDDEN; s++) begin : g_hidden_weights
    assign hidden_w_row[s] = hidden_weight_data[s*W_W +: W_W];
  end
  for (s = 0; s < N_OUTPUT; s++) begin : g_output_weights
    assign output_w_row[s] = output_weight_data[s*W_W +: W_W];
  end
  for (s = 0; s < N_OUTPUT; s++) begin : g_output_bus
    assign output_spike_bus[s] = output_spike[s];
  end
endgenerate

event_sched #(
    .N (N_INPUT_STREAMS)
) u_input_scheduler (
    .clk         (clk),
    .rst_n       (rst_n),
    .scan_req    (running && !scan_busy && !hidden_busy && !
      ev_valid),
    .ev_ready    (hidden_event_done),
    .next_t      (next_time),
    .valid       (stream_valid),
    .negative    (stream_negative),
    .scan_busy   (scan_busy),
    .scan_done   (scan_done),
    .ev_valid    (ev_valid),
    .ev_time     (ev_time),
    .ev_mask     (ev_mask),
    .ev_negative (ev_negative)
);

lif_layer #(
    .N (N_HIDDEN)
```

```
) u_hidden_layer (
    .clk           (clk),
    .rst_n         (rst_n),
    .clear         (clear_state),
    .ev_valid      (ev_valid),
    .dt            (ev_time[DT_W-1:0]),
    .ev_negative   (|ev_negative),
    .w_row         (hidden_w_row),
    .busy          (hidden_busy),
    .event_done    (hidden_event_done),
    .spk_valid     (hidden_spike),
    .spk_negative  (hidden_negative),
    .pot           (hidden_pot)
);

lif_layer_parallel #(
    .N (N_OUTPUT)
) u_output_layer (
    .clk           (clk),
    .rst_n         (rst_n),
    .clear         (clear_state),
    .ev_valid      (hidden_any_spike),
    .dt            ({{(DT_W-1){1'b0}}, 1'b1}),
    .ev_negative   (hidden_any_negative),
    .w_row         (output_w_row),
    .spk_valid     (output_spike),
    .spk_negative  (output_negative),
    .pot           (output_pot)
);

spike_counter u_output_counter (
    .clk     (clk),
    .rst_n   (rst_n),
    .clear   (clear_state),
    .spk_in  (output_spike_bus),
    .count   (output_count)
);

argmax u_argmax (
    .count (output_count),
    .digit (digit)
);

always_ff @(posedge clk or negedge rst_n) begin
  if (!rst_n) begin
    clear_state <= 1'b0;
    running     <= 1'b0;
    run_count   <= '0;
    stream_addr <= '0;
    done        <= 1'b0;
  end else begin
    clear_state <= 1'b0;
```

```
      done        <= 1'b0;
      if (!running) begin
        if (start) begin
          clear_state <= 1'b1;
          running     <= 1'b1;
          run_count   <= '0;
          stream_addr <= '0;
        end
      end else if (run_count == RUN_CYCLES - 1) begin
        running <= 1'b0;
        done    <= 1'b1;
      end else begin
        run_count <= run_count + 1'b1;
        if (hidden_event_done)
          stream_addr <= stream_addr + 1'b1;
      end
    end
  end

endmodule
```

Control flow:

```
start
  → clear_state = 1, running = 1, run_count = 0, stream_addr = 0
  → scan candidate events
  → hold earliest event
  → process hidden neurons 0..127
  → update output neurons
  → count output spikes
  → repeat until run_count == RUN_CYCLES - 1
  → done = 1, running = 0
  → digit = argmax(output_count)
```

Part VIII — Testbenches

31. `tb_time_integrator.sv`

Purpose: Tests the time integrator's delta accumulation, step enable, overflow detection, and clear semantics.

Test cases:

Label	Expected	Verifies
`initial reset`	`abs_time='0, is_overflow=0`	Reset state

Label	Expected	Verifies
`first step`	`abs_time=5, is_overflow=0`	Step with delta=5
`step disabled`	`abs_time=5`	Step=0 does not advance
`accumulated step`	`abs_time=7`	Step with delta=2
`overflow delta`	`is_overflow=1, abs_time=262`	Delta=0xFF advances time by 255
`overflow while stalled`	`is_overflow=1, abs_time=262`	Delta=0xFF does not advance when step=0
`clear takes priority`	`abs_time='0`	Clear overrides step
`step after clear`	`abs_time=1`	Step after clear
`asynchronous reset`	`abs_time='0`	Reset works
`step after reset`	`abs_time=3`	Step after reset

Full source code:

```
`timescale 1ns/1ps
import pkg_snn::*;

module tb_time_integrator;

  logic clk = 0;
  logic rst_n = 0;
  logic clear = 0;
  logic step = 0;
  logic [DT_W-1:0] delta = '0;
  logic [TIME_W-1:0] abs_time;
  logic is_overflow;

  integer checks = 0;

  time_integrator dut (.*);

  always #5 clk = ~clk;

  task automatic check_time(
      input logic [TIME_W-1:0] expected,
      input string label
  );
    begin
      checks = checks + 1;
      if (abs_time !== expected)
        $fatal(1, "%s: expected abs_time=%0d, got %0d",
               label, expected, abs_time);
    end
```

```
endtask

task automatic check_overflow(
    input logic expected,
    input string label
);
  begin
    checks = checks + 1;
    if (is_overflow !== expected)
      $fatal(1, "%s: expected is_overflow=%0b, got %0b",
             label, expected, is_overflow);
  end
endtask

initial begin
  @(posedge clk);
  #1;
  check_time('0, "initial reset");
  check_overflow(1'b0, "ordinary delta");

  @(negedge clk);
  rst_n = 1;
  step = 1;
  delta = 8'd5;
  @(posedge clk);
  #1 check_time(16'd5, "first step");

  @(negedge clk);
  step = 0;
  delta = 8'd7;
  @(posedge clk);
  #1 check_time(16'd5, "step disabled");

  @(negedge clk);
  step = 1;
  delta = 8'd2;
  @(posedge clk);
  #1 check_time(16'd7, "accumulated step");

  @(negedge clk);
  delta = 8'hff;
  check_overflow(1'b1, "overflow delta");
  @(posedge clk);
  #1 check_time(16'd262, "overflow advances time");

  @(negedge clk);
  step = 0;
  check_overflow(1'b1, "overflow indication while stalled");
  @(posedge clk);
  #1 check_time(16'd262, "overflow does not advance while
      stalled");
```

```
    @(negedge clk);
    clear = 1;
    step = 1;
    delta = 8'd4;
    @(posedge clk);
    #1 check_time('0, "clear takes priority over step");

    @(negedge clk);
    clear = 0;
    delta = 8'd1;
    @(posedge clk);
    #1 check_time(16'd1, "step after clear");

    @(negedge clk);
    rst_n = 0;
    step = 0;
    #1 check_time('0, "asynchronous reset");

    @(negedge clk);
    rst_n = 1;
    step = 1;
    delta = 8'd3;
    @(posedge clk);
    #1 check_time(16'd3, "step after reset");

    $display("PASS: tb_time_integrator (%0d checks)", checks);
    $finish;
  end

endmodule
```

32. `tb_event_sched.sv`

Purpose: Tests the event scheduler's minimum search, tie handling, and empty-scan behaviour.

Test cases:

Label	Input	Expected
`minimum with tied events`	Streams 1,2 at time 4; stream 0 at 9	`ev_valid=1`, `ev_time=4`, `ev_mask=0110` (streams 1,2), `ev_negative=0100` (stream 2 negative)
`no pending events`	All streams invalid	`ev_valid=0`, `scan_busy=0`

Label	Input	Expected
`maximum representable time`	Stream 3 at `0xFFFF`	`ev_valid=1, ev_time=0xFFFF, ev_mask=0001`

Full source code:

```
`timescale 1ns/1ps

module tb_event_sched;

  localparam int N = 4;

  logic clk = 0;
  logic rst_n = 0;
  logic scan_req = 0;
  logic ev_ready = 0;
  logic [15:0] next_t [N];
  logic valid [N];
  logic negative [N];
  logic scan_busy;
  logic scan_done;
  logic ev_valid;
  logic [15:0] ev_time;
  logic [N-1:0] ev_mask;
  logic [N-1:0] ev_negative;
  integer cycles;

  event_sched #(.N(N)) dut (.*);

  always #5 clk = ~clk;

  task automatic check_scan(
      input logic expected_valid,
      input logic [15:0] expected_time,
      input logic [N-1:0] expected_mask,
      input logic [N-1:0] expected_negative,
      input string label
  );
    begin
      @(negedge clk);
      scan_req = 1;
      @(negedge clk);
      scan_req = 0;

      cycles = 0;
      while (!scan_done && cycles < N + 3) begin
        @(posedge clk);
        #1 cycles = cycles + 1;
      end
```

```
    if (!scan_done)
      $fatal(1, "%s: scan did not complete", label);
    if (ev_valid !== expected_valid
        || ev_time !== expected_time
        || ev_mask !== expected_mask
        || ev_negative !== expected_negative)
      $fatal(1, "%s: valid=%0b time=%0d mask=%b negative=%b",
             label, ev_valid, ev_time, ev_mask, ev_negative);

    if (expected_valid) begin
      repeat (2) begin
        @(posedge clk);
        #1;
        if (!ev_valid || ev_time !== expected_time
            || ev_mask !== expected_mask
            || ev_negative !== expected_negative)
          $fatal(1, "%s: event changed before ready", label);
      end
      @(negedge clk);
      ev_ready = 1;
      @(negedge clk);
      ev_ready = 0;
      #1;
      if (ev_valid || scan_busy)
        $fatal(1, "%s: event was not released after ready",
       label);
    end else if (scan_busy) begin
      $fatal(1, "%s: empty scan left scheduler busy", label);
    end
  end
endtask

initial begin
  for (int i = 0; i < N; i++) begin
    next_t[i] = '0;
    valid[i] = 1'b0;
    negative[i] = 1'b0;
  end

  repeat (2) @(posedge clk);
  #1;
  if (scan_busy || ev_valid || scan_done)
    $fatal(1, "reset state is not idle");
  @(negedge clk);
  rst_n = 1;

  valid[0] = 1;
  next_t[0] = 16'd9;
  negative[0] = 1;
  valid[1] = 1;
  next_t[1] = 16'd4;
  negative[1] = 0;
```

```
    valid[2] = 1;
    next_t[2] = 16'd4;
    negative[2] = 1;
    check_scan(1, 16'd4, 4'b0110, 4'b0100, "minimum with tied
        events");

    for (int i = 0; i < N; i++) valid[i] = 1'b0;
    check_scan(0, '0, '0, '0, "no pending events");

    valid[3] = 1;
    next_t[3] = 16'hffff;
    negative[3] = 0;
    check_scan(1, 16'hffff, 4'b1000, '0, "maximum representable
        time");

    $display("PASS: tb_event_sched (3 scans)");
    $finish;
  end

endmodule
```

33. `tb_lif_neuron.sv`

Purpose: Tests the LIF neuron's positive-threshold, negative-threshold, decay, clear, and overflow behaviours.

Test cases:

Label	dt	input	Expected	Verifies
`positive threshold`	0	20	`(4, 1, 0)`	8 >> 0 + 20 = 28 ≥ 16; spike; pot = 12
`negative threshold option`	0	-16	`(0, 0, -1)`	With ENABLE_NEGATIVE_SPIKES=1, sum ≤ THETA_LOW=-16; negative spike
`clear priority`	0	20	`(0, 0, 0)`	Clear wins over event
`overflow does not update potential`	0xFF	0	`(12, 0, 0)`	Overflow does not update
`elapsed overflow time decays`	0xFF	0	`(12, 0, 0)`	Time advances, no neuron update; next event decays

Full source code:

```
`timescale 1ns/1ps
import pkg_snn::*;

module tb_lif_neuron;

  logic clk = 0;
  logic rst_n = 0;
  logic clear = 0;
  logic ev_valid = 0;
  logic [DT_W-1:0] dt = '0;
  logic signed [POT_W-1:0] synaptic_input = '0;

  logic spike_valid_positive;
  logic spike_negative_positive;
  logic signed [POT_W-1:0] pot_positive;
  logic spike_valid_bidirectional;
  logic spike_negative_bidirectional;
  logic signed [POT_W-1:0] pot_bidirectional;

  lif_neuron dut_positive (
      .clk,
      .rst_n,
      .clear,
      .ev_valid,
      .dt,
      .synaptic_input,
      .spike_valid   (spike_valid_positive),
      .spike_negative(spike_negative_positive),
      .pot           (pot_positive)
  );

  lif_neuron #(.ENABLE_NEGATIVE_SPIKES(1'b1)) dut_bidirectional (
      .clk,
      .rst_n,
      .clear,
      .ev_valid,
      .dt,
      .synaptic_input,
      .spike_valid   (spike_valid_bidirectional),
      .spike_negative(spike_negative_bidirectional),
      .pot           (pot_bidirectional)
  );

  always #5 clk = ~clk;

  task automatic check_state(
      input logic signed [POT_W-1:0] expected_positive_pot,
      input logic expected_positive_spike,
      input logic expected_positive_sign,
      input logic signed [POT_W-1:0] expected_bidirectional_pot,
```

```
    input logic expected_bidirectional_spike,
    input logic expected_bidirectional_sign,
    input string label
);
  begin
    if (pot_positive !== expected_positive_pot
        || spike_valid_positive !== expected_positive_spike
        || spike_negative_positive !== expected_positive_sign
        || pot_bidirectional !== expected_bidirectional_pot
        || spike_valid_bidirectional !==
      expected_bidirectional_spike
        || spike_negative_bidirectional !==
      expected_bidirectional_sign)
      $fatal(1, "%s: positive=(%0d,%0b,%0b) bidirectional=(%0d,
      %0b,%0b)",
             label,
             pot_positive, spike_valid_positive,
      spike_negative_positive,
             pot_bidirectional, spike_valid_bidirectional,
             spike_negative_bidirectional);
  end
endtask

task automatic apply_event(
    input logic [DT_W-1:0] event_dt,
    input logic signed [POT_W-1:0] event_input,
    input logic signed [POT_W-1:0] expected_positive_pot,
    input logic expected_positive_spike,
    input logic expected_positive_sign,
    input logic signed [POT_W-1:0] expected_bidirectional_pot,
    input logic expected_bidirectional_spike,
    input logic expected_bidirectional_sign,
    input string label
);
  begin
    @(negedge clk);
    dt = event_dt;
    synaptic_input = event_input;
    ev_valid = 1;
    @(posedge clk);
    #1 check_state(expected_positive_pot,
      expected_positive_spike,
                   expected_positive_sign,
      expected_bidirectional_pot,
                   expected_bidirectional_spike,
                   expected_bidirectional_sign, label);
    @(negedge clk);
    ev_valid = 0;
  end
endtask

initial begin
  @(posedge clk);
```

```
    #1 check_state('0, 1'b0, 1'b0, '0, 1'b0, 1'b0, "reset");

    @(negedge clk);
    rst_n = 1;

    apply_event(0, 20, 4, 1, 0, 4, 1, 0, "positive threshold");
    apply_event(1, 13, 15, 0, 0, 15, 0, 0, "decay before event");
    apply_event(0, 1, 0, 1, 0, 0, 1, 0, "threshold after decay");
    apply_event(0, -16, -16, 0, 0, 0, 1, 1, "negative threshold
        option");

    @(negedge clk);
    clear = 1;
    ev_valid = 1;
    synaptic_input = 20;
    @(posedge clk);
    #1 check_state('0, 1'b0, 1'b0, '0, 1'b0, 1'b0, "clear
        priority");
    @(negedge clk);
    clear = 0;
    ev_valid = 0;

    apply_event(0, 12, 12, 0, 0, 12, 0, 0, "subthreshold input");
    @(negedge clk);
    dt = DT_W'(OVERFLOW_CODE);
    ev_valid = 0;
    @(posedge clk);
    #1 check_state(12, 0, 0, 12, 0, 0, "overflow does not update
        potential");
    apply_event(DT_W'(OVERFLOW_CODE), 0, 0, 0, 0, 0, 0, 0,
                "elapsed overflow time decays on next event");

    @(negedge clk);
    rst_n = 0;
    #1 check_state('0, 1'b0, 1'b0, '0, 1'b0, 1'b0,
                   "asynchronous reset");

    $display("PASS: tb_lif_neuron (8 event checks)");
    $finish;
  end

endmodule
```

34. `tb_snn_top.sv`

Purpose: Full-image testbench. Loads one golden image and compares spike traces with Python. This is a **skeleton only** (TODO: skeleton only).

Test flow: 1. Apply reset (`rst_n=0` for 20ns) 2. Assert `start` for one clock edge 3. Wait for `done` 4. Display `predicted digit = %0d` 5. Finish

Full source code:

```
// tb_snn_top.sv
// Full-image testbench. Loads one golden image and compares spike
        traces with Python.
// TEMPLATE: skeleton only.
`timescale 1ns/1ps
import pkg_snn::*;

module tb_snn_top;

  logic clk = 0, rst_n = 0, start = 0;
  logic done;
  logic [3:0] digit;

  snn_top dut (.*);

  always #5 clk = ~clk;

  initial begin
    $dumpfile("../ignore/tb_snn_top.vcd");
    $dumpvars(0, tb_snn_top);
    #20 rst_n = 1;
    @(posedge clk); start = 1;
    @(posedge clk); start = 0;
    wait (done);
    $display("predicted digit = %0d", digit);
    $finish;
  end

endmodule
```

Part IX — Build, Project Config & Constraints

35. `run_sim.sh`

Purpose: Compiles and runs each testbench with Icarus Verilog.

Full source code:

```
#!/usr/bin/env bash
# run_sim.sh
# Compiles and runs each testbench with Icarus Verilog.
set -euo pipefail
cd "$(dirname "$0")/.."
mkdir -p ../ignore

SRC="rtl/util/pkg_snn.sv rtl/core/*.sv rtl/mem/*.sv rtl/util/
        argmax.sv rtl/top/*.sv"
```

```
echo "== tb_time_integrator =="
iverilog -g2012 -o "impl/tb_time_integrator.vvp" \
  rtl/util/pkg_snn.sv rtl/core/time_integrator.sv tb/
        tb_time_integrator.sv
vvp "impl/tb_time_integrator.vvp"

echo "== tb_event_sched =="
iverilog -g2012 -o "impl/tb_event_sched.vvp" \
  rtl/util/pkg_snn.sv rtl/core/event_sched.sv tb/tb_event_sched.sv
vvp "impl/tb_event_sched.vvp"

for tb in tb/tb_lif_neuron.sv tb/tb_snn_top.sv; do
  name=$(basename "$tb" .sv)
  echo "== $name =="
  iverilog -g2012 -o "impl/$name.vvp" $SRC "$tb"
  vvp "impl/$name.vvp"
done
```

Command breakdown: - `iverilog -g2012` — compile with SystemVerilog 2012 support - `-o impl/*.vvp` — output VVP (Verilog simulation) files - `vvp ...` — run the compiled simulation - The `cd` at the top navigates to the repository root regardless of the caller's directory

36. `build.tcl`

Purpose: Gowin EDA build flow template. TODOs are marked for the missing project/build commands.

Full source code:

```
# build.tcl
# Gowin EDA build flow for Tang Primer 20K.
# TEMPLATE: fill in the project settings from your Gowin EDA
        project.
set device "GW2A-LV18PG256C8/I7"
set top_module "snn_top"
set sources [glob -nocomplain rtl/util/pkg_snn.sv rtl/core/*.sv
        rtl/mem/*.sv rtl/util/argmax.sv rtl/top/*.sv]
set constraints constraints/tang_primer_20k.cst
# TODO: create project, add sources and constraints, run
        synthesis, PnR, bitstream to impl/
```

Missing elements (TODO): - Gowin project creation - Adding sources and constraints - Running synthesis - Running place-and-route - Generating the bitstream - Writing output to `impl/`

37. Project Configuration

`snn.gprj` (Gowin project XML): - Device: `GW2A-18C` (GW2A-LV18PG256C8/I7) - File list: 11 RTL files, 2 constraint files - Top module: `snn_top` (inferred from Gowin defaults)

`tang_primer_20k.cst` (pin constraints): - `clk` is assigned to pin `H11` with `IO_TYPE=LVCMOS33`, with a 3.3V VCCIO bank.

`tang_primer_20k.sdc` (timing constraints): - Onboard oscillator: 27 MHz (37.037 ns period, 0ns/18.518ns duty cycle).

38. Version Control

`.gitignore` — Excludes: - `impl/` (Gowin output) - `*.gprj.user`, `*.vvp`, `*.vcd` (IDE state, simulator output) - `data/` (MNIST, ROMs, checkpoints) - `ignore/` (generated artifacts)

Part X — Verification & Formats

39. Local Data Layout

The following paths are intentionally git-ignored (see `.gitignore`):

Path	Contents
`data/mnist/`	MNIST IDX images and labels
`data/streams/`	Generated event-stream ROM data
`data/weights/`	Checkpoints and generated weight ROM data
`ignore/`	Simulator traces and other generated artifacts

40. Memory Formats

File	Format	Width
`stream.mem`	Delta-time words, one per line	`DT_W = 8` bits
`hidden_w.mem`	Packed rows, 128 signed 4-bit weights	`N_NEURONS × W_W = 512` bits/row
`output_w.mem`	Packed rows, 10 signed 4-bit weights	`N_OUTPUT × W_W = 40` bits/row

41. Simulation & Test Commands

Command	Purpose
`bash src/scripts/run_sim.sh`	Run focused RTL regression with Icarus Verilog
`python -m unittest discover -s src/model -v`	Run Python unit tests
`verilator --lint-only --timing ...`	Lint with Verilator

42. Evidence Levels

Level	What it means
Unit-tested	Local function/module behavior passed focused test
Simulated	RTL regression completed in Icarus
Linted	Verilator elaborated the selected source set
Synthesized	Gowin generated a mapped design and reports
Routed/timing-reviewed	Place-and-route and timing reports were inspected
Hardware-validated	Bitstream was programmed and exercised on Tang Primer 20K

43. Known Gaps

1. The Python checkpoint is not converted into compatible ROM images.
2. Training and RTL threshold/weight units differ (`1.0` float vs `16` integer).
3. The top-level stream memories are not a complete per-row event-bank protocol.
4. The top-level run window is fixed at 1024 clocks.
5. The RTL top-level test is a control smoke test, not a Python golden-model comparison.
6. Gowin synthesis and board validation must be rerun for the optimized design.

44. Recommended Equivalence Path

1. Define one event and ROM schema.
2. Export a tiny hand-written image and weights.
3. Compare `snn_ref.run_image` with a Python serializer/deserializer round trip.
4. Feed the same image to RTL and compare event times, hidden spikes, output spikes, and counts.
5. Add overflow, zero-delta multiplicity, polarity, and simultaneous-stream cases.
6. Quantize a trained checkpoint and measure accuracy before FPGA synthesis.
7. Only then compare synthesized resources and validate the board.

Part XI — Top-Down RTL Workflow

45. Top-Level Interface

`snn_top` has one clocked transaction interface:

Port	Direction	Meaning
`clk`	input	FPGA system clock
`rst_n`	input	Active-low asynchronous reset
`start`	input	Starts one bounded inference run
`done`	output	One-cycle completion pulse
`digit[3:0]`	output	Current output spike-count winner

The board clock is constrained to 27 MHz in `tang_primer_20k.sdc`. The top-level does not currently have a valid/ready input image interface; the image/event data is expected to already exist in local ROM files.

46. Reset and Start

The top-level control process (the `always_ff` block at the bottom of `snn_top.sv`) handles reset and start:

On reset (`rst_n = 0`): - All state registers and outputs are reset to zero/ initial values

On start (`start = 1` when idle): - `clear_state = 1` (next clock) - `running = 1` - `run_count = 0` - `stream_addr = 0`

47. Memory Instances

The top-level instantiates two memory types:

Stream ROM: - File: `data/streams/stream.mem` - Reads one `DT_W`-bit (`8-bit`) word per address - The current top-level broadcasts the same `next_time` to all 28 stream slots (`next_time[s] = stream_data`)

Hidden-weight ROM: - File: `data/weights/hidden_w.mem` - Packed: `128 × 4 = 512` bits per input address (128 hidden neurons × 4-bit weights)

Output-weight ROM: - File: `data/weights/output_w.mem` - Packed: `10 × 4 = 40` bits per hidden-neuron address

48. Layer Processing

1. **Stream ROM:** `stream_addr` indexes the current stream word; `stream_data` holds the 8-bit delta-time value.
2. **Scheduler (`event_sched`):** Serial minimum search across 28 streams; produces `ev_valid`, `ev_time`, `ev_mask`, `ev_negative`.
3. **Hidden layer (`lif_layer`):** Processes the event across neurons 0–127; `event_done` gates the scheduler.
4. **Output layer (`lif_layer_parallel`):** Receives all 10 spikes from the hidden layer as a bus.
5. **Spike counter (`spike_counter`):** Counts one-cycle output-spike pulses per digit.
6. **Argmax (`argmax`):** Selects the digit with the highest count.

49. Run Completion

The top-level does not detect end-of-stream. Instead, it uses:

```
RUN_CYCLES = 1024
```

While `running`: - `run_count` increments each clock - `stream_addr` increments when `hidden_event_done` is asserted - When `run_count == RUN_CYCLES - 1`, `running` goes low, `done` pulses high

50. Simulation-to-Hardware Workflow

1. **RTL simulation:** `bash src/scripts/run_sim.sh` (Icarus Verilog)

2. **Lint:** `verilator --lint-only --timing ...`
3. **Gowin project:** `snn.gprj` + `build.tcl`
4. **Bitstream/board:** Gowin synthesis → PnR → bitstream → program Tang Primer 20K

51. Required Fixes for a Complete Implementation

1. Implement independent stream-bank state and end markers.
2. Align stream addresses and weight addresses with the accepted event.
3. Preserve simultaneous-stream and hidden-spike event multiplicity.
4. Define checkpoint quantization and packed ROM formats.
5. Replace the fixed 1024-cycle stop with a real end-of-image protocol.
6. Compare RTL traces against `snn_ref.run_image`.

Part XII — Appendix: Complete File Registry

Path	Lines	Category	Role
`README.md`	383	Doc	Root project README
`docs/Theory&Code/01-system-overview.md`	79	Doc	System overview
`docs/Theory&Code/02-theory.md`	137	Doc	Theory (differential-time encoding, LIF, training)
`docs/Theory&Code/03-python-model.md`	113	Doc	Python model guide
`docs/Theory&Code/04-rtl-architecture.md`	108	Doc	RTL architecture guide
`docs/Theory&Code/05-formats-and-verification.md`	102	Doc	Formats, verification, gaps
`docs/Theory&Code/06-top-down-workflow.md`	137	Doc	Top-down RTL workflow
`src/model/train_snn.py`	288	Code	Python MNIST trainer
	79	Code	

Path	Lines	Category	Role
src/model/encode_mnist.py			Differential-time event encoder
src/model/spike_events.py	26	Code	Event representation
src/model/snn_ref.py	227	Code	Integer reference model
src/model/quantize.py	20	Code	Weight quantizer (template)
src/model/export_mem.py	20	Code	Hex memory writer (template)
src/model/test_snn_model.py	117	Code	Model tests
src/model/test_train_snn.py	81	Code	Training tests
src/model/requirements-model.txt	4	Config	Python dependencies
src/rtl/util/pkg_snn.sv	24	Code	Central parameters
src/rtl/core/event_sched.sv	129	Code	Event scheduler
src/rtl/core/time_integrator.sv	29	Code	Time integrator
src/rtl/core/lif_neuron.sv	61	Code	LIF neuron
src/rtl/core/lif_layer.sv	113	Code	Hidden LIF layer
src/rtl/core/lif_layer_parallel.sv	46	Code	Output LIF layer
src/rtl/core/spike_counter.sv	24	Code	Spike counter
src/rtl/util/argmax.sv	17	Code	Winner selection
	20	Code	Stream ROM

Path	Lines	Category	Role
src/rtl/mem/stream_rom.sv			
src/rtl/mem/weight_rom.sv	22	Code	Weight ROM
src/rtl/top/snn_top.sv	202	Code	Top-level datapath
src/tb/tb_time_integrator.sv	109	Code	Time integrator testbench
src/tb/tb_event_sched.sv	112	Code	Scheduler testbench
src/tb/tb_lif_neuron.sv	136	Code	LIF neuron testbench
src/tb/tb_snn_top.sv	28	Code	Top-level testbench
src/scripts/run_sim.sh	38	Script	Icarus Verilog runner
src/scripts/build.tcl	9	Script	Gowin build template
snn.gprj	21	Config	Gowin project XML
snn.gprj.user	26	Config	Gowin project user settings
src/constraints/tang_primer_20k.cst	10	Config	Pin constraints
src/constraints/tang_primer_20k.sdc	2	Config	Timing constraints
.gitignore	31	Config	Git ignore rules

Generated from the `fpga-snn` repository (commit `f06fa7e1712462ff3affc4a04af4c4888e9378de`).